Angular de cero a experto

Guía detallada, con explicaciones y ejemplos, para aprender Angular desde cero hasta nivel profesional. Se organiza en cuatro partes y veinticuatro capítulos que conviene seguir en orden, porque cada uno se apoya en los anteriores. Está pensada para **Angular moderno (v22)**: componentes standalone, signals, el nuevo control flow y aplicaciones sin Zone.js.

> Consejo general: cuando busques tutoriales, descarta los que usen `NgModule`, `*ngIf`/`*ngFor` o `@Input()` como enfoque principal. Siguen funcionando, pero no son la forma actual de escribir Angular.

Índice

Parte I · Fundamentos

1. Fundamentos de TypeScript
2. Primeros pasos con Angular
3. Componentes
4. Pipes
5. Enlace de datos
6. Control de flujo en plantillas

Parte II · Desarrollo de aplicaciones

7. Signals y estado reactivo
8. Comunicación entre componentes
9. Servicios e inyección de dependencias
10. Enrutamiento y navegación
11. Formularios
12. Comunicación con el servidor

Parte III · Angular en profundidad

13. Programación reactiva con RxJS
14. Carga de datos asíncrona
15. Detección de cambios
16. Angular Material y CDK
17. Pruebas

Parte IV · Aplicaciones profesionales

18. Gestión del estado global
19. Renderizado en servidor
20. Optimización del rendimiento
21. Arquitectura de aplicaciones
22. Accesibilidad
23. Internacionalización y seguridad

24. Actualización y migraciones

Anexos

- Proyectos prácticos
- Recursos

---

1. Fundamentos de TypeScript

Angular está escrito en TypeScript y todo el código de una aplicación Angular también lo está. Este capítulo recoge lo imprescindible del lenguaje para entender el resto de la guía: tipos, interfaces, genéricos, clases, decoradores y código asíncrono (Promises, `fetch` y `async/await`).

Tipos básicos

TypeScript es JavaScript con tipos. Declarar el tipo de una variable permite que el editor y el compilador detecten errores antes de ejecutar el código: si intentas asignar un texto a una variable numérica, el error aparece mientras escribes. En la mayoría de los casos no hace falta escribir el tipo, porque TypeScript lo **infiere** a partir del valor inicial; conviene escribirlo explícitamente en parámetros de funciones, valores de retorno y cuando la variable empieza vacía.

Los tipos básicos son `string`, `number`, `boolean`, arrays (`string[]`), `null`, `undefined`, `any` (desactiva el tipado; evítalo) y `unknown` (como `any`, pero obliga a comprobar el tipo antes de usarlo).

```
let nombre: string = 'Ana';
let edad = 30;                  // inferido como number
let activo: boolean = true;
let etiquetas: string[] = ['angular', 'typescript'];

function sumar(a: number, b: number): number {
  return a + b;
}

edad = 'treinta';               // ❌ Error: string no es asignable a number
```

Interfaces

Una interfaz describe la **forma de un objeto**: qué propiedades tiene y de qué tipo es cada una. No genera código JavaScript; solo sirve para que el compilador compruebe que los objetos cumplen el contrato. En Angular las usarás constantemente para modelar los datos que llegan de una API (usuarios, productos, pedidos…).

```
interface Usuario {
  id: number;
  nombre: string;
}
```

```
const u: Usuario = { id: 1, nombre: 'Ana' };       // ✅
const v: Usuario = { id: 2 };                      // ❌ Falta 'nombre'
```

Las interfaces se pueden extender para reutilizar definiciones:

```
interface Empleado extends Usuario {
  departamento: string;
}
```

Propiedades opcionales y de solo lectura

Con `?` indicas que una propiedad **puede no existir**; TypeScript te obligará a comprobarla antes de usarla. Con `readonly` indicas que una propiedad **no se puede reasignar** después de crear el objeto, útil para identificadores o fechas de creación.

```
interface Usuario {
  id: number;
  nombre: string;
  email?: string;           // opcional
  readonly creadoEn: Date;  // no se puede reasignar
}

const u: Usuario = { id: 1, nombre: 'Ana', creadoEn: new Date() };
u.email?.toLowerCase();     // '?.' evita el error si email es undefined
u.creadoEn = new Date();    // ❌ Error: es de solo lectura
```

Tipos unión y literales

Un **tipo unión** (`A | B`) indica que un valor puede ser de varios tipos. Combinado con **literales** (valores concretos como `'admin'`) permite limitar una variable a un conjunto cerrado de opciones, de forma que el compilador avisa si escribes un valor que no existe. Se declaran con `type`.

```
type Rol = 'admin' | 'editor' | 'lector';
type Id = number | string;

let rol: Rol = 'admin';     // ✅
rol = 'superadmin';         // ❌ Error: no es un Rol válido

function mostrarId(id: Id) {
  if (typeof id === 'string') {
    return id.toUpperCase(); // aquí TypeScript sabe que es string
  }
  return id.toFixed(0);      // y aquí que es number
}
```

Este "estrechamiento" del tipo dentro de un `if` se llama **narrowing**.

Genéricos

Los genéricos permiten escribir funciones, interfaces o clases que funcionan con **cualquier tipo sin perder el tipado**. Se escriben con un parámetro de tipo entre `<>` (por convención `T`) que se concreta al usarlos. Angular los usa por todas partes: `signal<number>()`, `http.get<Usuario[]>()`, `input<string>()`.

```
interface RespuestaApi<T> {
  datos: T;
  total: number;
}

function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}

const r: RespuestaApi<Usuario[]> = { datos: [], total: 0 };
const n = primero([1, 2, 3]);       // n es number | undefined
const s = primero(['a', 'b']);      // s es string | undefined
```

Clases

Una clase es una plantilla para crear objetos con **propiedades y métodos**. En Angular, componentes, servicios, pipes y directivas son clases. El `constructor` se ejecuta al crear la instancia, y `this` se refiere a la propia instancia.

```
class Contador {
  valor = 0;

  incrementar(): void {
    this.valor++;
  }
}

const c = new Contador();
c.incrementar();
console.log(c.valor); // 1
```

Modificadores de acceso

Controlan desde dónde se puede usar un miembro de la clase:

- `public` (por defecto): accesible desde cualquier sitio.
- `private`: solo desde dentro de la clase.
- `protected`: desde la clase y sus subclases. En Angular se usa mucho para propiedades que la **plantilla** debe ver pero que no forman parte de la API pública del componente.

Escribir un modificador en un parámetro del constructor crea y asigna la propiedad automáticamente.

```
class Cuenta {
  private saldo = 0;

  constructor(public titular: string) {}  // crea this.titular

  ingresar(cantidad: number): void {
    if (cantidad <= 0) throw new Error('Cantidad inválida');
    this.saldo += cantidad;
  }
}

const c = new Cuenta('Ana');
c.titular;  // ✅
c.saldo;    // ❌ Error: es privada
```

Getters y setters

Un `get` expone un valor **calculado** como si fuera una propiedad; un `set` permite ejecutar lógica al asignar. Son útiles para dar acceso de solo lectura a un dato privado o para validar asignaciones.

```
class Cuenta {
  private saldo = 0;

  get saldoActual(): number {
    return this.saldo;
  }

  set limite(valor: number) {
    if (valor < 0) throw new Error('El límite no puede ser negativo');
    this._limite = valor;
  }
  private _limite = 0;
}

const c = new Cuenta();
console.log(c.saldoActual); // se usa sin paréntesis
c.limite = 500;
```

Utility types

Son tipos que TypeScript trae de serie para **crear un tipo nuevo a partir de otro** sin repetir la definición. Evitan duplicar interfaces casi iguales: por ejemplo, el tipo de un usuario al crearlo (sin `id`, porque lo genera el servidor) o al editarlo (con todo opcional). Los cuatro que más verás:

Partial

Convierte **todas las propiedades en opcionales**. Ideal para actualizaciones parciales, donde solo envías los campos que cambian.

```
type UsuarioParcial = Partial<Usuario>;

function actualizarUsuario(id: number, cambios: Partial<Usuario>) { /* ... */ }
actualizarUsuario(1, { nombre: 'Ana María' }); // no hace falta enviar el resto
```

Omit

Crea un tipo **quitando** las propiedades indicadas. Típico para formularios de alta, donde el `id` aún no existe.

```
type NuevoUsuario = Omit<Usuario, 'id' | 'creadoEn'>;

const nuevo: NuevoUsuario = { nombre: 'Luis', email: 'luis@mail.com' };
```

Pick

Lo contrario de `Omit`: crea un tipo **eligiendo solo** las propiedades indicadas. Útil para vistas resumidas.

```
type UsuarioResumen = Pick<Usuario, 'id' | 'nombre'>;

const fila: UsuarioResumen = { id: 1, nombre: 'Ana' };
```

Record

Crea un **diccionario** con claves de un tipo y valores de otro. Si las claves son un tipo unión, obliga a definir todas.

```
type Rol = 'admin' | 'editor' | 'lector';
type PermisosPorRol = Record<Rol, string[]>;

const permisos: PermisosPorRol = {
  admin: ['leer', 'escribir', 'borrar'],
  editor: ['leer', 'escribir'],
  lector: ['leer'],
}; // si faltara 'lector', daría error
```

Decoradores

Un decorador es una función que se escribe con `@` justo encima de una clase (o propiedad) y le **añade metadatos o comportamiento**. Angular usa los decoradores para saber qué es cada clase: `@Component` la convierte en componente, `@Service` en servicio, `@Pipe` en pipe y `@Directive` en directiva. No necesitas

escribir decoradores propios, pero sí entender que el objeto que reciben es la configuración de la clase que tienen debajo.

```
@Component({
  selector: 'app-saludo',          // etiqueta HTML del componente
  template: `<p>Hola</p>`,         // su vista
})
export class Saludo {}             // la clase que se configura

@Service()                          // la clase es un servicio disponible en toda
la app
export class Datos {}
```

Código asíncrono

La mayoría del código se ejecuta **en orden**, una línea detrás de otra, y cada línea termina al instante. Pero hay operaciones que tardan: pedir datos a un servidor, leer un archivo, esperar un temporizador. Si el programa se quedara parado esperando la respuesta, la página se congelaría: no se podría hacer clic, escribir ni hacer scroll.

Por eso JavaScript trabaja de forma **asíncrona**: lanza la operación lenta, sigue ejecutando el resto del código y, **cuando llega el resultado**, ejecuta lo que le hayamos indicado. Para trabajar así existen tres herramientas, que se explican a continuación: las **Promises**, la función `fetch` y la sintaxis `async/await`. En Angular se usa además una cuarta, los **Observables**, que se explican en el capítulo 12.

```
console.log('1. Antes del temporizador');

setTimeout(() => {
  console.log('3. Han pasado 2 segundos');
}, 2000);

console.log('2. Después del temporizador');

// Por consola sale 1, 2 y, dos segundos después, 3:
// el programa no se para a esperar al temporizador.
```

Promises

Una **Promise** (promesa) es un objeto que representa un valor que **todavía no existe, pero existirá en el futuro**. Es como el ticket que te dan al pedir en un restaurante de comida rápida: aún no tienes la comida, pero el ticket garantiza que te avisarán cuando esté lista.

Una promesa siempre está en uno de estos tres estados:

Estado	Significado
Pendiente (*pending*)	La operación todavía no ha terminado

Estado	Significado
Cumplida (*fulfilled*)	La operación ha terminado bien y tenemos el valor
Rechazada (*rejected*)	La operación ha fallado y tenemos el error

Una promesa **solo produce un valor**: cuando se cumple o se rechaza, ya no cambia.

Para hacer algo cuando la promesa termine se usan tres métodos, que se pueden encadenar:

- `.then(fn)`: se ejecuta cuando la promesa se cumple y recibe el valor.
- `.catch(fn)`: se ejecuta si la promesa se rechaza y recibe el error.
- `.finally(fn)`: se ejecuta siempre al terminar, haya ido bien o mal. Útil para ocultar un indicador de "cargando".

```
// Una promesa que se cumple al cabo de un segundo
const promesa = new Promise<string>((resolve, reject) => {
  setTimeout(() => resolve('¡Datos listos!'), 1000);
});

promesa
  .then((valor) => console.log(valor))          // ¡Datos listos!
  .catch((error) => console.error(error))
  .finally(() => console.log('Terminado'));
```

Normalmente no crearás promesas con `new Promise`: las recibirás de funciones del navegador o de librerías, como `fetch`.

fetch

`fetch` es la función del navegador para **hacer peticiones HTTP**, por ejemplo a una API. Recibe la URL y **devuelve una Promise** con la respuesta del servidor.

Obtener los datos requiere **dos pasos**, y cada uno devuelve una promesa:

1. `fetch(url)` se cumple cuando llegan las **cabeceras** de la respuesta. El resultado es un objeto `Response` con información como el código de estado.
2. `respuesta.json()` lee el **cuerpo** de la respuesta y lo convierte de texto JSON a un objeto de JavaScript.

```
fetch('https://pokeapi.co/api/v2/pokemon/pikachu')
  .then((respuesta) => respuesta.json())      // paso 2: convertir a objeto
  .then((datos) => console.log(datos.name))   // pikachu
  .catch((error) => console.error('Sin conexión', error));
```

Propiedades útiles del objeto `Response`:

Propiedad / método	Para qué sirve
`status`	Código de estado: 200 (bien), 404 (no encontrado), 500 (error del servidor)...
`ok`	`true` si el código está entre 200 y 299
`json()`	Lee el cuerpo como JSON y devuelve una promesa con el objeto
`text()`	Lee el cuerpo como texto plano

Por defecto `fetch` hace una petición **GET** (pedir datos). Para enviar datos se indica el método, las cabeceras y el cuerpo en un segundo parámetro:

```
fetch('https://api.ejemplo.com/tareas', {
  method: 'POST',                                    // crear un recurso
  headers: { 'Content-Type': 'application/json' },  // enviamos JSON
  body: JSON.stringify({ titulo: 'Estudiar Angular' }),
});
```

En Angular, para las peticiones se suele usar `HttpClient` (capítulo 12), que añade tipado, conversión automática a JSON e interceptores. Pero `fetch` es la base de todo y conviene entenderlo.

async/await

Encadenar muchos `.then()` se vuelve difícil de leer. `async/await` es una forma de usar promesas que permite escribir el código **de arriba abajo, como si fuera síncrono**:

- `async` delante de una función indica que dentro habrá esperas. Una función `async` **siempre devuelve una Promise**, aunque devuelvas un valor normal.
- `await` delante de una promesa significa "espera aquí a que se cumpla y dame su valor". Solo se puede usar dentro de una función `async`. Mientras espera, la función se pausa, pero el resto de la página sigue funcionando.

El mismo ejemplo anterior con `async/await`:

```
async function cargarPikachu() {
  const respuesta = await fetch('https://pokeapi.co/api/v2/pokemon/pikachu');
  const datos = await respuesta.json();
  console.log(datos.name); // pikachu
}
```

Con tipos, la función indica qué devuelve envuelto en `Promise<...>`:

```
interface PokemonRespuesta {
  id: number;
  name: string;
}
```

```
async function getPokemon(nombre: string): Promise<PokemonRespuesta> {
  const respuesta = await fetch(`https://pokeapi.co/api/v2/pokemon/${nombre}`);
  return respuesta.json();
}

async function mostrar() {
  const pokemon = await getPokemon('pikachu');
  console.log(pokemon.id); // 25
}
```

Gestionar errores con async/await

Con `async/await`, los errores se capturan con `try/catch`, igual que en el código normal: si una promesa con `await` se rechaza, la ejecución salta al `catch`.

Hay una trampa importante con `fetch`: **un 404 o un 500 no son un error para `fetch`**. La promesa solo se rechaza si **no llega ninguna respuesta** (sin conexión, servidor caído, URL imposible). Si el servidor responde, aunque sea con un error, la promesa se cumple. Por eso hay que comprobar `respuesta.ok` o `respuesta.status` y, si hace falta, lanzar el error nosotros con `throw`.

```
async function buscarPokemon(nombre: string) {
  try {
    const respuesta = await fetch(`https://pokeapi.co/api/v2/pokemon/${nombre}`);

    if (respuesta.status === 404) {
      console.log(`El Pokémon "${nombre}" no existe`);
      return;
    }
    if (!respuesta.ok) {
      throw new Error(`Error del servidor: ${respuesta.status}`); // salta al
catch
    }

    const datos = await respuesta.json();
    console.log(datos.name);
  } catch (error) {
    // Llega aquí si no hay conexión o si hemos lanzado el error con throw
    console.error('No se pudo cargar', error);
  } finally {
    console.log('Búsqueda terminada'); // se ejecuta siempre
  }
}
```

Esperar, no esperar y peticiones en paralelo

`await` **detiene la función** hasta que llega el resultado. Según lo que necesites, hay varias formas de lanzar una petición:

Quiero...	Cómo
Esperar al resultado antes de seguir	`const x = await promesa;`
Lanzarla y no esperar	`promesa;` sin `await` (con un `.catch()` para los errores)
Lanzarla y reaccionar cuando llegue, sin detenerme	`promesa.then(...)`
Lanzar varias a la vez y esperar a todas	Guardar las promesas y `await Promise.all(promesas)`

Un error habitual es poner `await` dentro de un bucle cuando las peticiones no dependen unas de otras: cada una espera a la anterior y todo va mucho más lento. Si son independientes, lánzalas todas sin `await` y espera al final con `Promise.all`, que devuelve un array con los resultados en el mismo orden:

```
const nombres = ['bulbasaur', 'charmander', 'squirtle'];

// ❌ Lento: una detrás de otra (si cada una tarda 200 ms, en total 600 ms)
for (const nombre of nombres) {
  const pokemon = await getPokemon(nombre);
  console.log(pokemon.name);
}

// ✅ Rápido: las tres a la vez (en total, unos 200 ms)
const promesas = nombres.map((nombre) => getPokemon(nombre)); // se lanzan aquí
const pokemons = await Promise.all(promesas);                 // se espera aquí
console.log(pokemons.map((p) => p.name));
```

Si falla una sola promesa, `Promise.all` falla entera. Si quieres los resultados aunque alguna falle, usa `Promise.allSettled`, que devuelve el estado de cada una.

Prácticas del capítulo 1

Práctica 1 · Básico — Modelar una biblioteca Crea una interfaz `Libro` con `id` (número), `titulo`, `autor`, `anio` (número), `genero` (que solo pueda ser `'novela'`, `'ensayo'` o `'poesia'`), `isbn` opcional y `fechaAlta` de solo lectura. Crea un array con tres libros y comprueba que TypeScript da error si escribes un género que no existe o si intentas cambiar `fechaAlta`.

Práctica 2 · Intermedio — Funciones genéricas y utility types

1. Escribe una función genérica `ultimo<T>(lista: T[]): T | undefined` que devuelva el último elemento de cualquier array.
2. Define el tipo `NuevoLibro` (sin `id` ni `fechaAlta`) con `Omit` y el tipo `CambiosLibro` con `Partial`.
3. Escribe `actualizarLibro(libro: Libro, cambios: CambiosLibro): Libro`, que devuelva un libro nuevo con los cambios aplicados sin modificar el original.

Práctica 3 · Reto — Clase con async/await Crea una clase `Pokedex` con un atributo privado `vistos: string[]`, un getter `total` que devuelva cuántos hay y un método `async registrar(nombre: string)`

que haga `fetch` a `https://pokeapi.co/api/v2/pokemon/{nombre}`. Si la respuesta es correcta, añade el nombre a `vistos`; si da 404, muestra por consola "No existe". Gestiona también el caso de que no haya conexión con `try/catch`.

---

2. Primeros pasos con Angular

Antes de escribir componentes conviene conocer la herramienta con la que se crean los proyectos, cómo se organizan sus archivos y cómo arranca una aplicación Angular.

Angular CLI

La CLI (Command Line Interface) es la herramienta oficial para **crear, generar, ejecutar, compilar, testear y actualizar** proyectos Angular. Genera el código siguiendo las convenciones oficiales, así que te ahorra errores y tiempo.

```
npm install -g @angular/cli
ng new mi-app
cd mi-app
ng serve           # servidor de desarrollo en http://localhost:4200
```

Comando	Para qué sirve
`ng generate component nombre` (`ng g c`)	Crear componente
`ng g service nombre` (`ng g s`)	Crear servicio
`ng g guard nombre`	Crear guard de rutas
`ng build`	Compilar para producción (carpeta `dist/`)
`ng test`	Ejecutar tests
`ng update`	Actualizar versión de Angular

`ng serve` recompila y recarga el navegador automáticamente cada vez que guardas un archivo.

Estructura del proyecto

Conocer dónde va cada cosa te permite orientarte en cualquier proyecto Angular. Los archivos más importantes son el punto de arranque (`main.ts`), el componente raíz (`app.ts`), la configuración global (`app.config.ts`) y las rutas (`app.routes.ts`).

```
src/
├── main.ts            ← punto de arranque
├── index.html         ← la única página HTML; contiene <app-root>
├── styles.css         ← estilos globales
└── app/
    ├── app.ts         ← componente raíz
    ├── app.html       ← su plantilla
```

```
      ├── app.config.ts  ← providers globales (router, http...)
      └── app.routes.ts  ← definición de rutas
```

Angular es una **SPA** (Single Page Application): solo existe `index.html` y Angular cambia su contenido sin recargar la página.

Arranque de la aplicación

`bootstrapApplication` arranca la aplicación a partir del componente raíz y de una configuración con los **providers** globales (router, cliente HTTP, etc.). Angular busca en `index.html` la etiqueta del componente raíz (`<app-root>`) y lo pinta ahí. Ya no hay `AppModule`.

```ts
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { App } from './app/app';
import { appConfig } from './app/app.config';

bootstrapApplication(App, appConfig);
```

```ts
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(),
  ],
};
```

Prácticas del capítulo 2

Práctica 1 · Básico — Primer proyecto Crea un proyecto nuevo llamado `mi-agenda` con `ng new mi-agenda` (crea la carpeta del proyecto con toda la estructura) y arráncalo con `ng serve` (lo abre en http://localhost:4200). Cambia el texto de `app.html` por un título con tu nombre y comprueba que el navegador se actualiza solo al guardar.

Práctica 2 · Intermedio — Explorar la estructura Responde, mirando los archivos del proyecto: ¿qué archivo arranca la aplicación?, ¿dónde se registran los providers globales?, ¿en qué etiqueta de `index.html` se pinta el componente raíz? Cambia el `<title>` de `index.html` y la etiqueta `selector` del componente raíz (y su uso en `index.html`) y comprueba que todo sigue funcionando.

Práctica 3 · Reto — Compilar para producción Ejecuta `ng build` (genera la versión optimizada en la carpeta `dist/`). Abre la carpeta generada y localiza el archivo JavaScript principal: ¿cuánto ocupa? Añade una librería grande o mucho texto al componente, vuelve a compilar y compara el tamaño. Explica con tus palabras por qué importa.

---

3. Componentes

Los componentes son los bloques con los que se construye cualquier interfaz en Angular. Este capítulo explica qué son, cómo se definen y cómo se combinan entre sí.

Componentes standalone

Un componente es la **pieza básica de la interfaz**: una clase con lógica, una plantilla HTML y estilos. Una aplicación Angular es un árbol de componentes que se anidan unos dentro de otros. "Standalone" significa que el componente declara él mismo lo que necesita en su array `imports` (otros componentes, pipes o directivas), sin depender de un módulo.

Propiedades principales del decorador:

- `selector`: la etiqueta con la que se usa (`<app-perfil />`).
- `imports`: lo que usa en su plantilla.
- `template` / `templateUrl`: plantilla en línea o en archivo aparte.
- `styles` / `styleUrl`: estilos en línea o en archivo. Son **encapsulados**: solo afectan a este componente.

```
import { Component } from '@angular/core';
import { Avatar } from './avatar';

@Component({
  selector: 'app-perfil',
  imports: [Avatar],
  templateUrl: './perfil.html',
  styleUrl: './perfil.css',
})
export class Perfil {
  nombre = 'Ana';
}
```

```
<!-- perfil.html -->
<app-avatar />
<h2>{{ nombre }}</h2>
```

Prácticas del capítulo 3

Práctica 1 · Básico — Tarjeta de presentación Genera un componente `tarjeta-perfil` con `ng generate component tarjeta-perfil` (crea la carpeta con la clase, la plantilla, los estilos y el test). Muestra en él tu nombre, una foto y una frase, y úsalo en el componente raíz.

Práctica 2 · Intermedio — Componentes anidados Crea un componente `cabecera` y otro `pie` y colócalos en el componente raíz alrededor de la tarjeta. Dale a cada uno estilos propios (por ejemplo, el mismo selector `h2` con colores distintos) y comprueba que los estilos de un componente no afectan a los demás.

Práctica 3 · Reto — Plantilla en línea y en archivo Convierte la cabecera a plantilla y estilos en línea (`template` y `styles`) y deja el pie con archivos separados (`templateUrl` y `styleUrl`). Escribe en un comentario cuándo usarías cada forma.

4. Pipes

Los pipes transforman datos para mostrarlos en la plantilla. Este capítulo cubre los pipes que trae Angular de serie y cómo crear los tuyos.

Pipes

Un pipe **transforma un valor para mostrarlo** en la plantilla sin modificar el dato original: formatear fechas, monedas, porcentajes, mayúsculas... Se aplica con `|` y puede recibir parámetros separados por `:`. Los pipes se pueden **encadenar**: la salida de uno es la entrada del siguiente, de izquierda a derecha.

Los pipes integrados se importan desde `@angular/common` y hay que añadirlos al array `imports` del componente que los usa.

```
import { DatePipe, CurrencyPipe, UpperCasePipe } from '@angular/common';

@Component({
  selector: 'app-pedido',
  imports: [DatePipe, CurrencyPipe, UpperCasePipe],
  template: `
    <h3>{{ cliente | uppercase }}</h3>
    <p>Fecha: {{ fecha | date:'longDate' }}</p>
    <p>Total: {{ total | currency:'EUR' }}</p>
    <p>{{ fecha | date:'EEEE' | uppercase }}</p>  <!-- encadenados: LUNES -->
  `,
})
export class Pedido {
  cliente = 'Ana';
  fecha = new Date();
  total = 49.9;
}
```

Pipes integrados

Angular trae de serie los pipes que cubren los formatos más habituales. Estos son todos los que incluye `@angular/common`:

Pipe	Clase a importar	Para qué sirve
`date`	`DatePipe`	Formatear fechas y horas
`uppercase`	`UpperCasePipe`	Texto en mayúsculas
`lowercase`	`LowerCasePipe`	Texto en minúsculas
`titlecase`	`TitleCasePipe`	Primera letra de cada palabra en mayúscula
`currency`	`CurrencyPipe`	Formatear importes con moneda
`number`	`DecimalPipe`	Formatear números y decimales

Pipe	Clase a importar	Para qué sirve
`percent`	`PercentPipe`	Formatear porcentajes
`json`	`JsonPipe`	Mostrar un objeto como JSON (depuración)
`slice`	`SlicePipe`	Recortar arrays o textos
`keyvalue`	`KeyValuePipe`	Recorrer las propiedades de un objeto
`async`	`AsyncPipe`	Mostrar el valor de un Observable o Promise
`i18nPlural`	`I18nPluralPipe`	Texto según cantidad (singular/plural)
`i18nSelect`	`I18nSelectPipe`	Texto según un valor concreto

Cómo se pasan los parámetros

Muchos pipes aceptan **parámetros** que modifican el resultado. Se escriben después del nombre del pipe, cada uno precedido de `:`, y **siempre en el mismo orden**. Todos son opcionales: si no indicas uno, se usa su valor por defecto. Para indicar un parámetro posterior sin cambiar uno anterior, hay que escribir igualmente el anterior (puedes pasar `undefined` para que use su valor por defecto).

```
{{ valor | pipe:parametro1:parametro2:parametro3 }}

<!-- Cambiar solo la zona horaria de date, manteniendo el formato por defecto -->
{{ fecha | date:undefined:'UTC' }}
```

En los apartados siguientes, cada pipe incluye una tabla con sus parámetros en el orden en que se escriben.

date

Formatea una fecha (`Date`, timestamp en milisegundos o texto ISO como `'2026-10-19T15:30:00'`).

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`format`	`'mediumDate'`	Formato predefinido o patrón propio
2	`timezone`	Zona del navegador	Zona horaria: `'UTC'`, `'+0200'`, `'-0500'`…
3	`locale`	`LOCALE_ID` de la app	Idioma con el que se escriben meses y días

Formatos predefinidos. Son atajos para los formatos más habituales. El resultado exacto depende del idioma; estos son con español y la fecha 19/10/2026 15:30:

Formato	Resultado
`short`	19/10/26, 15:30
`medium`	19 oct 2026, 15:30:00
`long`	19 de octubre de 2026, 15:30:00 GMT+2

Formato	Resultado
`full`	lunes, 19 de octubre de 2026, 15:30:00 (hora completa con zona)
`shortDate`	19/10/26
`mediumDate`	19 oct 2026
`longDate`	19 de octubre de 2026
`fullDate`	lunes, 19 de octubre de 2026
`shortTime`	15:30
`mediumTime`	15:30:00
`longTime`	15:30:00 GMT+2
`fullTime`	15:30:00 con la zona horaria completa

Patrones propios. Si ningún formato predefinido encaja, construyes el tuyo combinando símbolos. Cuantas más veces repites una letra, más largo es el resultado. El texto literal se escribe entre comillas simples.

Símbolo	Significado	Ejemplo
`yy` / `yyyy`	Año con 2 / 4 dígitos	26 / 2026
`M` / `MM`	Mes en número / con cero inicial	9 / 09
`MMM` / `MMMM`	Mes abreviado / completo	oct / octubre
`d` / `dd`	Día del mes / con cero inicial	5 / 05
`EEE` / `EEEE`	Día de la semana abreviado / completo	lun / lunes
`H` / `HH`	Hora 0–23 / con cero inicial	9 / 09
`h` / `hh`	Hora 1–12 / con cero inicial	3 / 03
`a`	AM / PM	p. m.
`m` / `mm`	Minutos / con cero inicial	5 / 05
`s` / `ss`	Segundos / con cero inicial	7 / 07
`w`	Semana del año	43
`z` / `Z`	Zona horaria	GMT+2 / +0200

```
<!-- fecha = new Date(2026, 9, 19, 15, 30) -->
<p>{{ fecha | date }}</p>                          <!-- 19 oct 2026 -->
<p>{{ fecha | date:'short' }}</p>                  <!-- 19/10/26, 15:30 -->
<p>{{ fecha | date:'fullDate' }}</p>               <!-- lunes, 19 de octubre de
2026 -->
<p>{{ fecha | date:'dd/MM/yyyy HH:mm' }}</p>       <!-- 19/10/2026 15:30 -->
<p>{{ fecha | date:"EEEE d 'de' MMMM" }}</p>       <!-- lunes 19 de octubre -->
<p>{{ fecha | date:'hh:mm a' }}</p>                <!-- 03:30 p. m. -->
```

```
<p>{{ fecha | date:'HH:mm':'UTC' }}</p>             <!-- 13:30 (hora en UTC) -->
<p>{{ fecha | date:'longDate':undefined:'en' }}</p> <!-- October 19, 2026 -->
```

Para no repetir el mismo formato en toda la app, puedes cambiar los valores por defecto del pipe:

```
// app.config.ts
providers: [
  { provide: DATE_PIPE_DEFAULT_OPTIONS, useValue: { dateFormat: 'dd/MM/yyyy',
timezone: '+0200' } },
]
```

uppercase, lowercase y titlecase

Cambian las mayúsculas y minúsculas de un texto. `titlecase` pone en mayúscula la primera letra de cada palabra y el resto en minúscula, útil para nombres que llegan de la API en formatos irregulares. **No tienen parámetros.**

```
<!-- nombre = 'aNa garcía lópez' -->
<p>{{ nombre | uppercase }}</p>   <!-- ANA GARCÍA LÓPEZ -->
<p>{{ nombre | lowercase }}</p>   <!-- ana garcía lópez -->
<p>{{ nombre | titlecase }}</p>   <!-- Ana García López -->
```

number

Formatea un número con separadores de miles y una cantidad concreta de decimales, redondeando cuando hace falta. Se explica antes que `currency` y `percent` porque los tres comparten el parámetro `digitsInfo`.

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`digitsInfo`	`'1.0-3'`	Cuántos dígitos enteros y decimales mostrar
2	`locale`	`LOCALE_ID` de la app	Idioma: decide los separadores (`1.234,5` o `1,234.5`)

Formato de `digitsInfo`. Es un texto con la forma `'A.B-C'`:

Parte	Significado	Por defecto
`A`	Mínimo de dígitos enteros (rellena con ceros a la izquierda)	1
`B`	Mínimo de decimales (rellena con ceros a la derecha)	0
`C`	Máximo de decimales (redondea si hay más)	3

Así, `'1.2-2'` significa "al menos un entero y exactamente dos decimales", y `'1.0-0'` significa "sin decimales".

```
<!-- valor = 3.14159; grande = 1500000; entero = 7 -->
<p>{{ valor | number }}</p>           <!-- 3,142 (por defecto, máx. 3 decimales) -
```

```
->
<p>{{ valor | number:'1.2-2' }}</p>   <!-- 3,14 -->
<p>{{ valor | number:'1.0-0' }}</p>   <!-- 3 -->
<p>{{ entero | number:'3.0-0' }}</p>  <!-- 007 -->
<p>{{ entero | number:'1.2-2' }}</p>  <!-- 7,00 -->
<p>{{ grande | number }}</p>          <!-- 1.500.000 -->
<p>{{ grande | number:'1.0-0':'en' }}</p> <!-- 1,500,000 -->
```

currency

Formatea un número como importe de dinero. La posición del símbolo (delante o detrás) y los separadores dependen del idioma configurado.

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`currencyCode`	`DEFAULT_CURRENCY_CODE` (`'USD'` si no lo cambias)	Moneda en código ISO 4217: `'EUR'`, `'USD'`, `'GBP'`…
2	`display`	`'symbol'`	Cómo se muestra la moneda (ver tabla siguiente)
3	`digitsInfo`	Según la moneda (normalmente `'1.2-2'`)	Mismo formato que en `number`
4	`locale`	`LOCALE_ID` de la app	Idioma

Valores de `display`:

Valor	Resultado con USD
`'symbol'`	US$ (símbolo sin ambigüedad)
`'symbol-narrow'`	$ (símbolo corto)
`'code'`	USD
Cualquier otro texto	Se muestra ese texto, por ejemplo `'dólares'`

```
<!-- precio = 1234.5 -->
<p>{{ precio | currency:'EUR' }}</p>                        <!-- 1.234,50 € -->
<p>{{ precio | currency:'EUR':'code' }}</p>                 <!-- 1.234,50 EUR -->
<p>{{ precio | currency:'USD' }}</p>                        <!-- 1.234,50 US$ -->
<p>{{ precio | currency:'USD':'symbol-narrow' }}</p>        <!-- 1.234,50 $ -->
<p>{{ precio | currency:'EUR':'symbol':'1.0-0' }}</p>       <!-- 1.235 € -->
<p>{{ precio | currency:'EUR':' euros' }}</p>               <!-- 1.234,50 euros -
->
```

percent

Multiplica el número por 100 y añade el símbolo `%`. Ojo: espera un valor entre 0 y 1 (`0.25` → 25 %); si tu dato ya viene como `25`, divídelo antes o usa `number` con el símbolo a mano.

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`digitsInfo`	`'1.0-0'`	Mismo formato que en `number`
2	`locale`	`LOCALE_ID` de la app	Idioma

```
<!-- progreso = 0.256 -->
<p>{{ progreso | percent }}</p>           <!-- 26 % -->
<p>{{ progreso | percent:'1.1-1' }}</p>  <!-- 25,6 % -->
<p>{{ progreso | percent:'1.2-2' }}</p>  <!-- 25,60 % -->
```

json

Convierte un objeto en texto JSON con indentación. No se usa para mostrar datos al usuario, sino para **depurar**: ver de un vistazo qué contiene una variable. Combínalo con `<pre>` para que se respeten los saltos de línea. **No tiene parámetros.**

```
<pre>{{ usuario | json }}</pre>
<!--
{
  "id": 1,
  "nombre": "Ana"
}
-->
```

slice

Devuelve una parte de un array o de un texto, igual que el método `slice` de JavaScript. No modifica el original.

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`start`	Obligatorio	Índice donde empieza (incluido)
2	`end`	Hasta el final	Índice donde termina (no incluido)

Los índices **negativos** cuentan desde el final: `-3` significa "los tres últimos".

```
<!-- letras = ['a', 'b', 'c', 'd', 'e']; descripcion es un texto largo -->
<p>{{ letras | slice:0:3 }}</p>          <!-- a,b,c (los tres primeros) -->
<p>{{ letras | slice:2 }}</p>            <!-- c,d,e (desde el índice 2) -->
<p>{{ letras | slice:-2 }}</p>           <!-- d,e (los dos últimos) -->
<p>{{ descripcion | slice:0:100 }}…</p>  <!-- primeros 100 caracteres -->
```

```
@for (p of productos | slice:0:3; track p.id) {
  <li>{{ p.nombre }}</li>
}
```

keyvalue

Convierte un objeto (o un `Map`) en un array de pares `{ key, value }`, de modo que se puede **recorrer con `@for`**.

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`compareFn`	Orden por clave	Función que decide el orden de los pares; `null` mantiene el orden original

La función de comparación recibe dos pares y devuelve un número negativo si el primero va antes, positivo si va después y `0` si da igual, como en `Array.sort`.

```
stock = { sillas: 12, mesas: 4, lamparas: 0 };

porCantidad = (a: KeyValue<string, number>, b: KeyValue<string, number>) =>
b.value - a.value;
```

```
@for (item of stock | keyvalue; track item.key) {
  <p>{{ item.key }}: {{ item.value }}</p>   <!-- lamparas, mesas, sillas
(alfabético) -->
}

@for (item of stock | keyvalue:porCantidad; track item.key) {
  <p>{{ item.key }}: {{ item.value }}</p>   <!-- sillas, mesas, lamparas (de más a
menos) -->
}

@for (item of stock | keyvalue:null; track item.key) {
  <p>{{ item.key }}: {{ item.value }}</p>   <!-- sillas, mesas, lamparas (orden
original) -->
}
```

async

Se **suscribe a un Observable o Promise** y muestra su último valor; cuando el componente se destruye, cancela la suscripción automáticamente. Mientras no llega el primer valor devuelve `null`, por eso se suele combinar con `@if (... ; as nombre)`. **No tiene parámetros.**

Era muy común antes de los signals; hoy lo verás sobre todo en código existente, porque `toSignal` o `httpResource` suelen ser más cómodos. Aun así, conviene conocerlo.

```
export class Usuarios {
  usuarios$ = inject(UsuariosApi).listar();
}
```

```
@if (usuarios$ | async; as usuarios) {
  @for (u of usuarios; track u.id) { <li>{{ u.nombre }}</li> }
} @else {
  <p>Cargando…</p>
}
```

i18nPlural

Elige un texto según una **cantidad**, para escribir correctamente singular y plural. En el texto elegido, `#` se sustituye por el número.

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`pluralMap`	Obligatorio	Objeto con un texto para cada caso
2	`locale`	`LOCALE_ID` de la app	Idioma, que decide las reglas de plural

Claves que admite el objeto:

Clave	Cuándo se usa
`'=0'`, `'=1'`, `'=5'`…	Cuando la cantidad es exactamente ese número (tiene prioridad)
`'one'`, `'few'`, `'many'`…	Categorías de plural del idioma (en español solo existen `one` y `other`)
`'other'`	Cualquier otro caso. Inclúyela siempre como respaldo

```
mensajes = signal(3);
mapaMensajes = {
  '=0': 'No tienes mensajes',
  '=1': 'Tienes un mensaje',
  'other': 'Tienes # mensajes',
};
```

```
<p>{{ mensajes() | i18nPlural:mapaMensajes }}</p>  <!-- Tienes 3 mensajes -->
```

i18nSelect

Elige un texto según un **valor concreto** (un estado, un tipo, un género). Es como un `@switch` en una sola línea.

Orden	Parámetro	Valor por defecto	Para qué sirve
1	`mapping`	Obligatorio	Objeto con un texto para cada valor posible

Si el valor no está en el objeto, se usa la clave `'other'`; si tampoco existe, el resultado es un texto vacío.

```
estado = 'enviado';
mapaEstados = {
  pendiente: 'Pendiente de envío',
  enviado: 'En camino',
  entregado: 'Entregado',
  other: 'Estado desconocido',
};
```

```
<p>{{ estado | i18nSelect:mapaEstados }}</p>  <!-- En camino -->
```

Configurar el idioma de los pipes

Los pipes de fecha, número, moneda y porcentaje formatean según el **idioma (locale)** de la aplicación. Por defecto es inglés de EE. UU. (`1,234.50`, `Oct 19, 2026`). Para usar el formato español hay que registrar los datos del idioma y declararlo como `LOCALE_ID`.

```
// app.config.ts
import { LOCALE_ID } from '@angular/core';
import { registerLocaleData } from '@angular/common';
import localeEs from '@angular/common/locales/es';

registerLocaleData(localeEs);

export const appConfig: ApplicationConfig = {
  providers: [
    { provide: LOCALE_ID, useValue: 'es' },
    { provide: DEFAULT_CURRENCY_CODE, useValue: 'EUR' }, // moneda por defecto
  ],
};
```

Pipes propios

Cuando necesitas una transformación que Angular no trae (iniciales de un nombre, tiempo relativo como "hace 5 minutos", formatear un teléfono, traducir un estado), puedes **crear tu propio pipe**. Así la lógica de formato queda en un único sitio, se reutiliza en cualquier plantilla y se testea fácilmente.

Crear un pipe

Un pipe es una clase con el decorador `@Pipe`, donde `name` es el nombre que se usará en la plantilla, y que implementa la interfaz `PipeTransform`. Esta obliga a definir el método `transform`, que recibe el valor de entrada y devuelve el valor transformado. Se genera con la CLI y, como cualquier pipe, se añade a los `imports` del componente que lo use.

```
ng generate pipe iniciales
```

```
// iniciales-pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'iniciales' })
export class InicialesPipe implements PipeTransform {
  transform(nombre: string): string {
    return nombre
      .split(' ')
      .filter(p => p.length > 0)
      .map(p => p[0])
      .join('')
      .toUpperCase();
  }
}
```

```
@Component({
  selector: 'app-avatar',
  imports: [InicialesPipe],
  template: `<div class="avatar">{{ nombre() | iniciales }}</div>`, // muestra:
AGL
})
export class Avatar {
  nombre = input.required<string>(); // 'Ana García López'
}
```

Pipes con parámetros

Los parámetros que se escriben tras `:` en la plantilla llegan como **argumentos adicionales** de `transform`, en el mismo orden. Dales valores por defecto para que sean opcionales.

```
@Pipe({ name: 'truncar' })
export class TruncarPipe implements PipeTransform {
  transform(texto: string, limite = 50, sufijo = '…'): string {
    return texto.length > limite ? texto.slice(0, limite).trimEnd() + sufijo :
texto;
  }
}
```

```
<p>{{ descripcion | truncar }}</p>              <!-- límite 50, sufijo … -->
<p>{{ descripcion | truncar:20 }}</p>          <!-- límite 20 -->
<p>{{ descripcion | truncar:20:' [ver más]' }}</p>
```

Pipes puros e impuros

Por defecto los pipes son **puros**: Angular solo vuelve a ejecutar `transform` cuando **cambia el valor de entrada o un parámetro** (por referencia, en el caso de objetos y arrays). Eso los hace muy eficientes, y por eso son una buena alternativa a llamar funciones desde la plantilla.

Un pipe **impuro** (`pure: false`) se ejecuta en **cada revisión de la vista**, aunque nada haya cambiado. Solo tiene sentido si el resultado depende de algo externo al valor de entrada (como la hora actual) o si necesitas detectar cambios dentro de un array mutado. Úsalos con mucha precaución, porque pueden ralentizar la aplicación.

```
// Puro (por defecto): solo se recalcula si cambia 'estado'
@Pipe({ name: 'estadoPedido' })
export class EstadoPedidoPipe implements PipeTransform {
  private textos: Record<string, string> = {
    pendiente: 'Pendiente', enviado: 'En camino', entregado: 'Entregado',
  };
  transform(estado: string): string {
    return this.textos[estado] ?? 'Desconocido';
  }
}

// Impuro: el resultado depende de la hora actual, no solo de la fecha recibida
@Pipe({ name: 'haceTiempo', pure: false })
export class HaceTiempoPipe implements PipeTransform {
  transform(fecha: Date): string {
    const minutos = Math.floor((Date.now() - fecha.getTime()) / 60000);
    if (minutos < 1) return 'Ahora mismo';
    if (minutos < 60) return `Hace ${minutos} min`;
    return `Hace ${Math.floor(minutos / 60)} h`;
  }
}
```

Pipes que usan servicios

Un pipe puede **inyectar servicios** con `inject()`, igual que un componente. Es útil cuando el formato depende de datos de la aplicación: el idioma activo, una tabla de traducciones o la configuración del usuario.

```
@Pipe({ name: 'nombreCategoria' })
export class NombreCategoriaPipe implements PipeTransform {
  private categorias = inject(CategoriasService);

  transform(id: number): string {
```

```
    return this.categorias.porId(id)?.nombre ?? 'Sin categoría';
  }
}
```

```
<p>{{ producto.categoriaId | nombreCategoria }}</p>
```

Testear un pipe

Como un pipe es una clase con un método, se testea de forma directa: se crea una instancia y se comprueba la salida de `transform` con distintas entradas. Solo hace falta `TestBed` si el pipe inyecta servicios.

```
describe('TruncarPipe', () => {
  const pipe = new TruncarPipe();

  it('no recorta textos cortos', () => {
    expect(pipe.transform('Hola', 10)).toBe('Hola');
  });

  it('recorta y añade el sufijo', () => {
    expect(pipe.transform('Angular es genial', 7)).toBe('Angular…');
  });
});
```

Prácticas del capítulo 4

Práctica 1 · Básico — Ficha de producto En un componente con un objeto `producto` (`nombre`, `precio`, `descuento` entre 0 y 1, `fechaLlegada`), muestra: el nombre en mayúsculas, el precio en euros, el descuento como porcentaje y la fecha en formato `dd/MM/yyyy`. Configura el idioma español para que el precio salga como `1.234,50 €`.

Práctica 2 · Intermedio — Parámetros de los pipes Con el mismo producto, muestra el precio sin decimales, con el código `EUR` en lugar del símbolo, y la fecha en formato largo con el día de la semana (`lunes, 19 de octubre`). Muestra los 3 primeros elementos de un array de etiquetas con `slice` y recorre un objeto de stock por tallas con `keyvalue`, ordenado de mayor a menor stock.

Práctica 3 · Reto — Pipes propios Crea tres pipes: `iniciales` ("Ana García" → "AG"), `truncar` con un parámetro de longitud y otro opcional de sufijo, y `tiempoLectura`, que reciba un texto y devuelva "X min de lectura" calculando 200 palabras por minuto. Escribe un test para cada uno.

---

5. Enlace de datos

El enlace de datos (data binding) conecta la clase del componente con su plantilla: muestra valores, asigna propiedades, reacciona a eventos y sincroniza campos de formulario.

Interpolación

La interpolación `{{ }}` **muestra en la vista el valor** de una propiedad o expresión de la clase. Se actualiza automáticamente cuando el valor cambia. Dentro se pueden poner expresiones sencillas (operaciones, llamadas a signals, ternarios), pero no lógica compleja.

```
<p>Hola, {{ nombre }}</p>
<p>Tienes {{ mensajes.length }} mensajes</p>
<p>{{ edad >= 18 ? 'Mayor de edad' : 'Menor de edad' }}</p>
```

Property binding

Con `[propiedad]="expresión"` **asignas el valor de una expresión a una propiedad** de un elemento HTML o de un componente hijo. A diferencia de la interpolación, no convierte el valor en texto: pasa el valor real (un booleano, un objeto...).

```
<img [src]="urlFoto" [alt]="nombre" />
<button [disabled]="cargando">Guardar</button>
<app-tarjeta [producto]="productoSeleccionado" />
```

Event binding

Con `(evento)="sentencia"` **reaccionas a eventos** del DOM (click, input, submit, keyup...) o de componentes hijos. `$event` contiene el objeto del evento. También puedes filtrar teclas: `(keyup.enter)`.

```
<button (click)="guardar()">Guardar</button>
<input (input)="buscar($any($event.target).value)" />
<input (keyup.enter)="enviar()" />
```

Two-way binding

`[(propiedad)]` combina property y event binding: el valor fluye **en las dos direcciones**. Si cambia la clase, cambia la vista; si el usuario modifica la vista, cambia la clase. La sintaxis `[( )]` se conoce como "banana in a box".

En realidad es solo **azúcar sintáctico**: Angular lo traduce a un property binding `[x]` más un event binding `(xChange)`. Por eso funciona con cualquier componente que tenga una propiedad `x` y un evento `xChange`.

```
<app-estrellas [(valor)]="puntuacion" />

<!-- equivale exactamente a: -->
<app-estrellas [valor]="puntuacion()" (valorChange)="puntuacion.set($event)" />
```

Con signals, si enlazas un `WritableSignal`, Angular lo lee y le hace `set` automáticamente. En el lado del hijo, la propiedad bidireccional se declara con `model()` (se explica en detalle en el capítulo 8).

Para que se vea completo, estos son los dos componentes del ejemplo. El **hijo** es un componente de valoración con estrellas: recibe la puntuación y la cambia cuando el usuario pulsa una estrella.

```
// estrellas.ts (componente hijo)
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-estrellas',
  template: `
    @for (n of [1, 2, 3, 4, 5]; track n) {
      <button type="button" (click)="valor.set(n)">
        {{ n <= valor() ? '★' : '☆' }}
      </button>
    }
  `,
})
export class Estrellas {
  // model() crea a la vez la propiedad "valor" y el evento "valorChange"
  valor = model(0);
}
```

El **padre** guarda la puntuación en un signal y la enlaza con `[( )]`. Cuando el hijo hace `valor.set(n)`, el signal del padre se actualiza solo; y si el padre cambia `puntuacion`, las estrellas del hijo también cambian.

```
// valoracion.ts (componente padre)
import { Component, signal } from '@angular/core';
import { Estrellas } from './estrellas';

@Component({
  selector: 'app-valoracion',
  imports: [Estrellas],
  template: `
    <app-estrellas [(valor)]="puntuacion" />
    <p>Has puntuado: {{ puntuacion() }} de 5</p>
    <button type="button" (click)="puntuacion.set(0)">Borrar</button>
  `,
})
export class Valoracion {
  puntuacion = signal(3); // empieza con 3 estrellas marcadas
}
```

Fíjate en que en `[(valor)]="puntuacion"` se escribe el signal **sin paréntesis**: se pasa el signal entero para que Angular pueda leerlo y escribirlo, no solo su valor actual.

Dónde se usa hoy:

- **Componentes propios tipo control**: selectores, valoraciones, interruptores, pestañas. Es el caso principal.

- **Componentes de librerías** que exponen valores bidireccionales (por ejemplo, `[(selectedIndex)]` en las pestañas de Material).
- **Campos de formulario sueltos** con `[(ngModel)]`, que se explica a continuación.

ngModel

`ngModel` es una directiva que conecta un **campo de formulario nativo** (`input`, `textarea`, `select`, checkbox, radio) con una propiedad o signal del componente. Los elementos HTML no tienen una propiedad y un evento con el patrón `x`/`xChange`, así que no admiten `[( )]` directamente; `ngModel` hace de puente: escribe el valor en el campo y escucha los cambios del usuario.

Requiere importar `FormsModule` en el componente.

```
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-saludo',
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="nombre" placeholder="Tu nombre" />
    <p>Hola, {{ nombre() }}</p>
  `,
})
export class Saludo {
  nombre = signal('');
}
```

ngModel con distintos tipos de campo

`ngModel` se adapta al tipo de campo: en un texto enlaza un `string`, en un checkbox un `boolean`, en un `select` el valor de la opción elegida, y en un `type="number"` un número. Con `[ngValue]` en las opciones de un `select` puedes enlazar **objetos completos**, no solo textos.

```
@Component({
  imports: [FormsModule],
  template: `
    <textarea [(ngModel)]="comentario"></textarea>

    <label>
      <input type="checkbox" [(ngModel)]="aceptaCondiciones" /> Acepto las
condiciones
    </label>

    <input type="number" [(ngModel)]="cantidad" min="1" />

    <label><input type="radio" name="envio" value="normal" [(ngModel)]="envio" />
Normal</label>
    <label><input type="radio" name="envio" value="urgente" [(ngModel)]="envio" />
Urgente</label>
```

```
    <select [(ngModel)]="ciudad">
      @for (c of ciudades; track c.id) {
        <option [ngValue]="c">{{ c.nombre }}</option>
      }
    </select>
    <p>Has elegido: {{ ciudad()?.nombre }}</p>
  `,
})
export class Pedido {
  comentario = signal('');
  aceptaCondiciones = signal(false);
  cantidad = signal(1);
  envio = signal<'normal' | 'urgente'>('normal');
  ciudades = [{ id: 1, nombre: 'Bilbao' }, { id: 2, nombre: 'Madrid' }];
  ciudad = signal<{ id: number; nombre: string } | null>(null);
}
```

Reaccionar a cambios con ngModelChange

Si además de guardar el valor necesitas **ejecutar algo cada vez que cambia** (lanzar una búsqueda, validar, guardar un borrador), separa el binding en sus dos partes y usa el evento `(ngModelChange)`. Con signals, a menudo es más limpio usar un `computed` o un `effect` sobre el propio signal.

```
<input [ngModel]="termino()" (ngModelChange)="buscar($event)" />
```

```
termino = signal('');

buscar(texto: string) {
  this.termino.set(texto);
  console.log('Buscando', texto);
}
```

Con `ngModelOptions` puedes cambiar **cuándo** se actualiza el valor; por ejemplo, al salir del campo en lugar de con cada tecla:

```
<input [(ngModel)]="email" [ngModelOptions]="{ updateOn: 'blur' }" />
```

Validación con ngModel

`ngModel` reconoce los **atributos de validación de HTML** (`required`, `minlength`, `maxlength`, `min`, `max`, `pattern`, `email`) y calcula el estado del campo. Para consultarlo, se crea una **variable de plantilla** con `#nombre="ngModel"`, que da acceso a `valid`, `invalid`, `touched`, `dirty` y `errors`.

Dentro de una etiqueta `<form>`, cada campo con `ngModel` **necesita el atributo** `name`, porque Angular agrupa los campos por nombre. Con `#f="ngForm"` accedes al estado del formulario completo.

```
<form #f="ngForm" (ngSubmit)="enviar()">
  <input name="email" type="email" [(ngModel)]="email" required email
#emailCtrl="ngModel" />

  @if (emailCtrl.touched && emailCtrl.errors?.['required']) {
    <small>El email es obligatorio</small>
  } @else if (emailCtrl.touched && emailCtrl.errors?.['email']) {
    <small>El email no es válido</small>
  }

  <button [disabled]="f.invalid">Enviar</button>
</form>
```

El componente solo necesita importar `FormsModule`, guardar el valor en un signal y tener el método que se ejecuta al enviar. Toda la validación está en la plantilla.

```
import { Component, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-suscripcion',
  imports: [FormsModule],
  templateUrl: './suscripcion.html',
})
export class Suscripcion {
  email = signal('');

  enviar() {
    console.log('Suscrito con', this.email());
    this.email.set(''); // vaciamos el campo
  }
}
```

A este enfoque, en el que la lógica del formulario vive en la plantilla, se le llama **formularios template-driven**.

Cuándo usar ngModel y cuándo no

`ngModel` es ideal para **campos sueltos o formularios muy pequeños**: un buscador, un filtro, un checkbox de "mostrar completados", un selector de orden. Es rápido de escribir y, con signals, muy limpio.

Para **formularios con muchos campos, validaciones cruzadas o lógica dinámica** (añadir y quitar campos), es mejor usar formularios reactivos o signal forms (se explican en el capítulo 11): tienen tipado completo, la validación está en la clase y son más fáciles de testear. Lo que ha perdido terreno no es el two-way binding, sino construir formularios grandes a base de `ngModel`.

```
<!-- ✅ Buen uso: filtro sencillo -->
<input [(ngModel)]="filtro" placeholder="Filtrar tareas" />
<label><input type="checkbox" [(ngModel)]="soloPendientes" /> Solo
pendientes</label>

@for (t of tareasFiltradas(); track t.id) { <li>{{ t.titulo }}</li> }
```

```
filtro = signal('');
soloPendientes = signal(false);

tareasFiltradas = computed(() =>
  this.tareas()
    .filter(t => !this.soloPendientes() || !t.hecha)
    .filter(t => t.titulo.toLowerCase().includes(this.filtro().toLowerCase()))
);
```

Binding de clases y estilos

Permite **añadir clases CSS o estilos de forma condicional**. `[class.nombre]` añade la clase si la expresión es verdadera; `[style.propiedad]` asigna un valor de estilo (con unidad opcional).

```
<li [class.activo]="esActivo" [class.error]="tieneError">Elemento</li>
<div [style.color]="colorTexto" [style.width.px]="ancho"></div>
<div [class]="{ destacado: esDestacado, oculto: !visible }"></div>
```

Prácticas del capítulo 5

Práctica 1 · Básico — Los cuatro bindings Crea un componente con un campo de texto y un botón. Muestra el texto escrito con interpolación, desactiva el botón con property binding cuando el campo esté vacío, y al pulsarlo (event binding) muestra un mensaje de saludo.

Práctica 2 · Intermedio — Formulario de preferencias con ngModel Crea un formulario con `[(ngModel)]` y signals que tenga: nombre (texto), edad (número), idioma (`select` con tres opciones), recibir noticias (checkbox) y tema claro u oscuro (radio). Debajo, muestra un resumen que se actualice mientras el usuario escribe. Aplica la clase `oscuro` al contenedor con `[class.oscuro]` cuando se elija el tema oscuro.

Práctica 3 · Reto — Componente con model() Crea un componente `contador-cantidad` con botones − y + que use `model()` para su valor y no permita bajar de 0 ni pasar de un máximo recibido con `input()`. Úsalo en un componente padre con `[(cantidad)]="unidades"` y muestra en el padre el precio total según las unidades.

---

6. Control de flujo en plantillas

El control de flujo decide qué partes de la plantilla se muestran, cuántas veces se repiten y cuándo se cargan. Usa bloques que empiezan por `@` y no necesitan imports.

@if

Muestra u oculta un bloque de la plantilla según una condición. Admite `@else if` y `@else`. Sustituye a `*ngIf` y no necesita imports. Lo que está dentro de un bloque no visible **no existe en el DOM**, no solo está oculto.

```
@if (usuario) {
  <p>Hola, {{ usuario.nombre }}</p>
} @else if (cargando) {
  <p>Cargando…</p>
} @else {
  <p>Inicia sesión</p>
}
```

@for

Repite un bloque por cada elemento de una lista. `track` es **obligatorio**: le dice a Angular cómo identificar cada elemento para que, cuando la lista cambie, solo cree, mueva o elimine los elementos afectados en lugar de repintar todo. Usa un id único siempre que puedas. `@empty` define qué mostrar si la lista está vacía.

Variables disponibles: `$index`, `$first`, `$last`, `$even`, `$odd`, `$count`.

```
<ul>
  @for (tarea of tareas; track tarea.id; let i = $index, ultimo = $last) {
    <li [class.ultima]="ultimo">{{ i + 1 }}. {{ tarea.titulo }}</li>
  } @empty {
    <li>No hay tareas</li>
  }
</ul>
```

@switch

Elige un bloque entre varios según el valor de una expresión. Es más legible que encadenar muchos `@else if` cuando comparas una misma variable con valores concretos.

```
@switch (rol) {
  @case ('admin')  { <app-panel-admin /> }
  @case ('editor') { <app-panel-editor /> }
  @default         { <app-panel-lector /> }
}
```

@defer

Carga una parte de la plantilla **de forma diferida**: el código de los componentes que contiene se separa en otro archivo y solo se descarga cuando se cumple el disparador. Reduce el tamaño inicial de la aplicación y acelera la primera carga. Es una de las herramientas de rendimiento más potentes de Angular.

Bloques auxiliares:

- `@placeholder`: qué se ve antes de empezar a cargar.
- `@loading`: qué se ve mientras se descarga.
- `@error`: qué se ve si falla.

Disparadores: `on viewport` (al entrar en pantalla), `on idle` (cuando el navegador está libre; es el valor por defecto), `on interaction` (al hacer clic o enfocar), `on hover`, `on timer(2s)`, `when condición`.

```
@defer (on viewport) {
  <app-grafica-pesada />
} @placeholder {
  <div class="hueco">La gráfica aparecerá aquí</div>
} @loading (minimum 300ms) {
  <p>Cargando gráfica…</p>
} @error {
  <p>No se pudo cargar</p>
}
```

Prácticas del capítulo 6

Práctica 1 · Básico — Lista de la compra Crea un array de productos con `id`, `nombre` y `comprado`. Recórrelo con `@for` (con su `track`), muestra los comprados tachados con `@if` o una clase, y muestra "La lista está vacía" con `@empty`.

Práctica 2 · Intermedio — Semáforo con @switch Crea un componente con un estado `'verde' | 'ambar' | 'rojo'` y un botón que lo cambie al siguiente. Con `@switch`, muestra un mensaje distinto para cada estado ("Pasa", "Precaución", "Para"). Numera las filas de la práctica anterior con `$index` y resalta la primera y la última con `$first` y `$last`.

Práctica 3 · Reto — Carga diferida Crea un componente `grafica` que simule ser pesado (por ejemplo, con un array de 5000 elementos). Colócalo al final de una página larga dentro de un `@defer (on viewport)` con `@placeholder` y `@loading`. Abre las herramientas del navegador (F12 → Red) y comprueba que su código solo se descarga al hacer scroll hasta él.

Repaso de la Parte I

Antes de pasar a la Parte II deberías poder hacer todo lo siguiente sin consultar apuntes.

- ☐ Escribo interfaces, uniones y genéricos sin dudar
- ☐ Uso `Partial`, `Omit`, `Pick` y `Record` cuando corresponde
- ☐ Creo componentes con la CLI y los uso en otros componentes
- ☐ Domino los tipos de binding y los pipes integrados
- ☐ Uso `@if`, `@for` (con `track`) y `@switch`
- ☐ Sé cuándo aplicar `@defer`

7. Signals y estado reactivo

Los signals son el sistema de reactividad de Angular moderno: guardan el estado de la aplicación y hacen que la vista se actualice sola cuando cambia. Son la base de todo lo que viene después.

Qué es un signal

Un signal es un **contenedor de un valor que avisa cuando cambia**. Cuando una plantilla, un `computed` o un `effect` lee un signal, Angular registra esa dependencia; al cambiar el valor, Angular actualiza **solo lo que depende de él**. Es la base de la reactividad en Angular moderno y lo que permite prescindir de Zone.js.

Un signal se **lee llamándolo como función**: `contador()`. Ese paréntesis es lo que permite a Angular saber quién lo está leyendo.

```
import { signal } from '@angular/core';

const contador = signal(0);
console.log(contador()); // 0
```

signal: set y update

Un `signal()` es **escribible**. Tiene dos formas de cambiar su valor:

- `set(valor)`: reemplaza el valor.
- `update(fn)`: calcula el nuevo valor a partir del anterior. Úsalo cuando el nuevo valor dependa del actual.

```
const contador = signal(0);

contador.set(5);              // 5
contador.update(v => v + 1);  // 6

@Component({
  template: `
    <p>{{ contador() }}</p>
    <button (click)="contador.update(v => v + 1)">+1</button>
    <button (click)="contador.set(0)">Reset</button>
  `,
})
export class Contador {
  contador = signal(0);
}
```

computed

Crea un signal **de solo lectura cuyo valor se deriva de otros signals**. Se recalcula automáticamente cuando cambia alguna de sus dependencias, y además es perezoso (solo calcula si alguien lo lee) y se cachea (si las

dependencias no cambian, devuelve el valor guardado). Regla práctica: todo lo que se pueda calcular a partir de otro estado debe ser un `computed`, no un signal aparte que mantengas sincronizado a mano.

```
const precio = signal(100);
const cantidad = signal(2);

const total = computed(() => precio() * cantidad()); // 200
cantidad.set(3);                                     // total pasa a 300 solo
total.set(10);                                       // ❌ Error: es de solo
lectura
```

effect

Ejecuta una función **cada vez que cambia algún signal que lee**. Sirve para **efectos secundarios**: guardar en `localStorage`, escribir logs, sincronizar con una librería externa (un mapa, un gráfico). No lo uses para copiar un signal en otro: para eso existe `computed`. Debe crearse en un contexto de inyección (normalmente el constructor o como propiedad de clase).

```
export class Preferencias {
  tema = signal<'claro' | 'oscuro'>('claro');

  constructor() {
    effect(() => {
      localStorage.setItem('tema', this.tema()); // se ejecuta al cambiar tema
    });
  }
}
```

linkedSignal

Es un signal **derivado de otro pero que también se puede modificar**. Se reinicia con el valor calculado cada vez que cambia su origen, y entre medias puedes hacerle `set`. El caso típico: una selección por defecto que el usuario puede cambiar, pero que debe reiniciarse cuando cambia la lista de opciones.

```
const opciones = signal(['Madrid', 'Bilbao', 'Sevilla']);
const seleccionada = linkedSignal(() => opciones()[0]); // 'Madrid'

seleccionada.set('Bilbao');                  // el usuario elige
opciones.set(['Valencia', 'Zaragoza']);      // seleccionada vuelve a 'Valencia'
```

Actualizaciones inmutables

Un signal detecta un cambio cuando **recibe una referencia nueva**. Si modificas un array u objeto por dentro (`push`, asignar una propiedad), la referencia es la misma y Angular no se entera. Por eso los cambios en arrays y objetos se hacen creando copias con el operador spread (`...`), `map` o `filter`.

```
const tareas = signal<Tarea[]>([]);

// ❌ Mal: muta el array original, la vista no se actualiza
tareas().push(nuevaTarea);

// ✅ Bien: crea un array nuevo
tareas.update(lista => [...lista, nuevaTarea]);
tareas.update(lista => lista.filter(t => t.id !== id));
tareas.update(lista => lista.map(t => t.id === id ? { ...t, hecha: true } : t));
```

Prácticas del capítulo 7

Práctica 1 · Básico — Contador con signals Crea un contador con botones +1, −1 y reset usando `signal`, `set` y `update`. Añade un `computed` que indique si el número es par o impar y otro que diga si es positivo, negativo o cero.

Práctica 2 · Intermedio — Lista de tareas inmutable Guarda una lista de tareas en un `signal<Tarea[]>`. Implementa añadir, borrar y marcar como hecha, siempre de forma inmutable (`[...lista]`, `filter`, `map`). Añade `computed` para el número de tareas pendientes y para el porcentaje completado. Comprueba qué pasa si usas `push` en lugar de crear un array nuevo.

Práctica 3 · Reto — Persistencia con effect y selección con linkedSignal Guarda automáticamente la lista de tareas en `localStorage` con un `effect` y cárgala al iniciar. Añade un filtro de categorías y una `categoriaSeleccionada` con `linkedSignal` que vuelva a la primera categoría cuando cambie la lista de categorías.

---

8. Comunicación entre componentes

Una aplicación es un árbol de componentes que necesitan pasarse datos y avisarse de lo que ocurre. Este capítulo explica las tres herramientas para hacerlo: `input()`, `output()` y `model()`.

input()

Declara una **propiedad que el componente recibe de su padre**. El resultado es un signal de solo lectura, así que se lee con `()` y se puede usar dentro de `computed`. `input.required()` obliga al padre a pasarlo (error de compilación si falta); `input(valorPorDefecto)` lo hace opcional.

```
@Component({
  selector: 'app-tarjeta-producto',
  template: `
    <h3>{{ producto().nombre }}</h3>
    <p>{{ precioConIva() }} €</p>
  `,
})
export class TarjetaProducto {
  producto = input.required<Producto>();
  iva = input(0.21);
```

```
  precioConIva = computed(() => this.producto().precio * (1 + this.iva()));
}
```

```
<app-tarjeta-producto [producto]="p" />
<app-tarjeta-producto [producto]="p" [iva]="0.10" />
```

output()

Declara un **evento que el componente emite hacia su padre**. El hijo llama a `emit(valor)` y el padre lo escucha con event binding, recibiendo el valor en `$event`. Así el hijo comunica que algo ha pasado sin saber qué hará el padre con ello.

```
export class TarjetaProducto {
  producto = input.required<Producto>();
  anadir = output<number>();
}
```

```
<!-- plantilla del hijo -->
<button (click)="anadir.emit(producto().id)">Añadir</button>

<!-- plantilla del padre -->
@for (p of productos(); track p.id) {
  <app-tarjeta-producto [producto]="p" (anadir)="alCarrito($event)" />
}
```

model()

Crea un **binding bidireccional entre padre e hijo**: el padre pasa un valor y el hijo puede modificarlo, de modo que el cambio llega de vuelta al padre. Es un input y un output en uno. Se usa en componentes tipo control de formulario: selectores, valoraciones, interruptores.

```
@Component({
  selector: 'app-estrellas',
  template: `
    @for (n of [1, 2, 3, 4, 5]; track n) {
      <button (click)="valor.set(n)">{{ n <= valor() ? '★' : '☆' }}</button>
    }
  `,
})
export class Estrellas {
  valor = model(0);
}
```

```
<!-- padre: puntuacion es un signal -->
<app-estrellas [(valor)]="puntuacion" />
<p>Has puntuado: {{ puntuacion() }}</p>
```

Prácticas del capítulo 8

Práctica 1 · Básico — Tarjeta con input() Crea un componente `tarjeta-contacto` que reciba un contacto obligatorio con `input.required<Contacto>()` y un `input` opcional `destacado` (por defecto `false`). Muestra varias tarjetas desde el padre con `@for`.

Práctica 2 · Intermedio — Comunicación hacia el padre con output() Añade a la tarjeta dos botones, "Favorito" y "Eliminar", que emitan eventos con `output<number>()` enviando el `id` del contacto. En el padre, elimina el contacto de la lista o márcalo como favorito según el evento recibido.

Práctica 3 · Reto — Selector reutilizable con model() Crea un componente `selector-color` que muestre cinco círculos de colores y use `model<string>()` para el color elegido. Úsalo en un formulario de "Nueva nota" con `[(color)]` y pinta la nota del color seleccionado. Añade un `computed` en el hijo que calcule si el color es claro u oscuro para elegir el color del texto.

---

9. Servicios e inyección de dependencias

Los servicios contienen la lógica y el estado que no pertenecen a un componente concreto, y la inyección de dependencias es el mecanismo con el que Angular los crea y los reparte.

Servicios

Un servicio es una clase que contiene **lógica o estado que no pertenece a un componente concreto**: llamadas a la API, autenticación, carrito de la compra, preferencias. Separar esta lógica de los componentes los mantiene pequeños y permite que varios compartan los mismos datos.

Los servicios se marcan con el decorador `@Service()`. Con él, Angular registra el servicio automáticamente en toda la aplicación y crea **una única instancia** que comparten todos los que lo usen. Es lo que genera la CLI:

```
ng generate service carrito
```

Este comando crea el archivo `carrito.ts` con la clase del servicio ya decorada, y su archivo de tests `carrito.spec.ts`.

```
import { Service, computed, signal } from '@angular/core';

@Service()
export class Carrito {
  private readonly _items = signal<ItemCarrito[]>([]);

  readonly total = computed(() =>
```

```
    this._items().reduce((s, i) => s + i.precio * i.cantidad, 0)
  );

  anadir(item: ItemCarrito) {
    this._items.update(lista => [...lista, item]);
  }

  vaciar() {
    this._items.set([]);
  }
}
```

@Service y @Injectable

Hasta hace poco, los servicios se declaraban con `@Injectable({ providedIn: 'root' })`. `@Service()` hace lo mismo con menos código y es la forma recomendada en Angular moderno. Aun así, `@Injectable` sigue funcionando y lo verás en casi todos los proyectos y tutoriales existentes, así que conviene reconocerlo.

```
// Forma actual
@Service()
export class Carrito {}

// Forma anterior, equivalente
@Injectable({ providedIn: 'root' })
export class Carrito {}
```

Hay una diferencia importante: un servicio con `@Service()` **solo puede pedir sus dependencias con `inject()`**, no a través del constructor. Si copias un ejemplo antiguo que use el constructor, el compilador dará un error.

```
@Service()
export class ProductosApi {
  constructor(private http: HttpClient) {}  // ❌ Error: no se permite en
@Service
}

@Service()
export class ProductosApi {
  private http = inject(HttpClient);        // ✅ Correcto
}
```

Si necesitas generar un servicio con el formato antiguo (por ejemplo, para mantener el estilo de un proyecto existente), la CLI tiene una opción para ello:

```
ng generate service carrito --injectable
```

Este comando genera el servicio con `@Injectable({ providedIn: 'root' })` en lugar de `@Service()`.

private y readonly en los servicios

En los ejemplos de servicios verás que muchas propiedades llevan `private`, `readonly` o las dos. No es decoración: protegen el estado del servicio, que es compartido por toda la aplicación. Si cualquier componente pudiera cambiarlo de cualquier manera, sería muy difícil encontrar quién ha provocado un error.

- `private` (explicado en el capítulo 1, "Modificadores de acceso") hace que la propiedad **solo se pueda usar dentro del servicio**. Los componentes no pueden ni leerla ni modificarla. Se usa para las piezas internas: el `HttpClient`, la URL base o el signal escribible con los datos.
- `readonly` impide **reasignar** la propiedad: no se puede hacer `this.total = otraCosa`, ni desde fuera ni desde dentro. Se usa en propiedades que se crean una vez y no deben sustituirse nunca, como los signals. Ojo: `readonly` no impide cambiar el *valor* del signal con `set` o `update`; impide sustituir el signal entero por otro, lo que rompería a todos los componentes que lo estaban leyendo.
- **Sin** `private` (público) quedan solo las propiedades y métodos que los componentes deben usar: es la "API" del servicio.
- El **guion bajo** delante del nombre (`_items`) es solo una convención para indicar "esta es la versión interna"; no tiene efecto en el código.

```
@Service()
export class Carrito {
  // Interno: solo el servicio lo usa y nadie puede sustituirlo
  private readonly http = inject(HttpClient);
  private readonly _items = signal<ItemCarrito[]>([]);

  // Público y de solo lectura: los componentes pueden leerlo, pero no reasignarlo
  readonly total = computed(() => this._items().reduce((s, i) => s + i.precio *
i.cantidad, 0));

  // Público: la única forma de modificar el carrito desde fuera
  anadir(item: ItemCarrito) {
    this._items.update(lista => [...lista, item]);
  }
}
```

Lo que ocurre si un componente intenta saltarse estas reglas:

```
export class Tienda {
  private carrito = inject(Carrito);

  probar() {
    this.carrito.total();              // ✅ leer lo público
    this.carrito.anadir(item);         // ✅ usar un método público
    this.carrito._items.set([]);       // ❌ Error: _items es privado
    this.carrito.total = signal(0);    // ❌ Error: total es readonly
  }
}
```

TypeScript detecta estos errores al escribir el código, antes de ejecutar la aplicación. Así el servicio decide **cómo** cambian sus datos, y si hay que añadir una regla (por ejemplo, no permitir cantidades negativas) basta con cambiar el método `anadir`.

En los componentes se aplica la misma idea con `protected`: lo que solo usa la plantilla se marca como `protected`, y lo que no usa nadie de fuera, como `private`.

inject()

La **inyección de dependencias** es el mecanismo por el que Angular crea las instancias de los servicios y se las entrega a quien las pide. Tú no haces `new Carrito()`; lo pides con `inject(Carrito)` y Angular te da la instancia correspondiente. Ventajas: no acoplas el código a cómo se crean las dependencias, compartes instancias y en los tests puedes sustituirlas por versiones falsas.

`inject()` solo funciona en un **contexto de inyección**: al declarar propiedades de clase, en el constructor, o en funciones como guards e interceptores.

```
@Component({
  selector: 'app-resumen',
  template: `Total: {{ carrito.total() }} €`,
})
export class Resumen {
  protected carrito = inject(Carrito);
}
```

Encapsular el estado con asReadonly

Si un servicio expone un signal escribible, cualquier componente podría modificarlo de cualquier forma y sería difícil saber quién cambió qué. El patrón recomendado es mantener el signal escribible **privado** y exponer una versión **de solo lectura** con `asReadonly()`. Los cambios solo se hacen a través de métodos del servicio, que concentran las reglas de negocio.

La diferencia con `readonly` es importante: `readonly` impide sustituir el signal por otro, pero un componente aún podría hacer `items.set(...)`. `asReadonly()` devuelve un signal que **no tiene `set` ni `update`**, así que desde fuera solo se puede leer.

```
@Service()
export class Carrito {
  private readonly _items = signal<ItemCarrito[]>([]);
  readonly items = this._items.asReadonly(); // los componentes solo leen

  anadir(item: ItemCarrito) {                // y modifican por aquí
    if (item.cantidad <= 0) return;
    this._items.update(lista => [...lista, item]);
  }
}
```

Ámbitos de provisión

Dónde se **provee** un servicio determina cuántas instancias hay y quién las comparte:

- `@Service()`: una única instancia (singleton) para toda la app. Es lo más común.
- `providers` en un componente: una instancia nueva por cada instancia del componente y sus hijos. Útil para estado local de un formulario complejo.
- `providers` en una ruta: una instancia compartida por esa rama de rutas.

Para los dos últimos casos, el servicio se declara con `@Service({ autoProvided: false })`. Así Angular **no** lo registra automáticamente en toda la app, y solo existe donde lo añadas en un array `providers`.

```
// Singleton global
@Service()
export class Auth {}

// Solo existe donde se añada a "providers"
@Service({ autoProvided: false })
export class EstadoEditor {}

// Una instancia por cada <app-editor>
@Component({
  selector: 'app-editor',
  providers: [EstadoEditor],
  template: `...`,
})
export class Editor {}

// Compartida dentro de /admin
{ path: 'admin', providers: [AdminApi], loadChildren: () =>
import('./admin/admin.routes') }
```

InjectionToken

Permite **inyectar valores que no son clases**: una URL, un objeto de configuración, una función. Creas un token, le asignas un valor en los providers y lo inyectas donde lo necesites. Así la configuración no está escrita a fuego dentro del código y es fácil cambiarla en tests o entornos.

```
export const API_URL = new InjectionToken<string>('API_URL');

// app.config.ts
providers: [{ provide: API_URL, useValue: 'https://api.ejemplo.com' }]

// en cualquier servicio
export class ProductosApi {
  private apiUrl = inject(API_URL);
}
```

Prácticas del capítulo 9

Práctica 1 · Básico — Servicio de favoritos Crea con `ng generate service favoritos` (genera la clase del servicio con `@Service()`) un servicio que guarde una lista de nombres en un signal privado. Expón la lista con `asReadonly()` y los métodos `anadir`, `quitar` y `esFavorito`. Úsalo desde dos componentes distintos y comprueba que ambos ven los mismos datos.

Práctica 2 · Intermedio — Encapsulación En el servicio anterior, añade un `computed` público `total` y una regla en `anadir`: no se pueden repetir nombres ni tener más de 10 favoritos. Desde un componente, intenta modificar el signal privado y reasignar `total`, y anota los errores que da TypeScript. Explica por qué es mejor así.

Práctica 3 · Reto — Ámbitos de provisión y tokens Crea un servicio `EstadoFormulario` con `@Service({ autoProvided: false })` y añádelo en los `providers` de un componente. Coloca dos instancias de ese componente en la página y comprueba que cada una tiene su propio estado. Crea además un `InjectionToken TITULO_APP` con el nombre de la aplicación y muéstralo en la cabecera.

---

10. Enrutamiento y navegación

El router permite que una aplicación de una sola página tenga varias pantallas, cada una con su URL, y controla cómo se navega entre ellas.

Definición de rutas

El router asocia **URLs con componentes**: al navegar a `/productos`, Angular pinta el componente correspondiente sin recargar la página. Las rutas se definen en un array; cada una tiene un `path` y lo que debe mostrar. Los segmentos con `:` son **parámetros** (`:id`), `title` cambia el título de la pestaña y `**` captura cualquier URL no definida (página 404). El orden importa: Angular usa la primera ruta que coincide.

```
// app.routes.ts
export const routes: Routes = [
  { path: '', component: Inicio, title: 'Inicio' },
  { path: 'productos', component: ListaProductos, title: 'Productos' },
  { path: 'productos/:id', component: DetalleProducto },
  { path: 'antigua', redirectTo: 'productos' },
```

```
  { path: '**', component: NoEncontrado },
];
```

Lazy loading de rutas

Cargar una ruta de forma perezosa significa que su código **solo se descarga cuando el usuario navega a ella**. Así la carga inicial es más ligera. `loadComponent` carga un componente; `loadChildren` carga un grupo entero de rutas (una sección como el panel de administración).

```
export const routes: Routes = [
  {
    path: 'perfil',
    loadComponent: () => import('./perfil/perfil').then(m => m.Perfil),
  },
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
  },
];
```

```
// admin/admin.routes.ts
export const ADMIN_ROUTES: Routes = [
  { path: '', component: PanelAdmin },
  { path: 'usuarios', component: GestionUsuarios },
];
```

Configurar el router

El router se activa registrándolo en los providers con `provideRouter`. Se le pueden añadir funcionalidades extra con funciones `with…`. La más útil es `withComponentInputBinding()`, que se explica a continuación.

```
// app.config.ts
import { provideRouter, withComponentInputBinding } from '@angular/router';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withComponentInputBinding())],
};
```

Parámetros de ruta como inputs

Con `withComponentInputBinding()` activado, los **parámetros de la URL, los query params y los datos de resolvers llegan al componente como `input()`**, con el mismo nombre. Es mucho más sencillo que leerlos desde `ActivatedRoute`, y al ser signals puedes reaccionar a ellos con `computed`.

```
// Ruta: productos/:id   →   URL: /productos/42?vista=detalle
export class DetalleProducto {
  id = input.required<string>();   // '42'
  vista = input<string>();         // 'detalle'
}
```

routerLink y router-outlet

`<router-outlet />` marca **dónde se pinta el componente de la ruta activa**. `routerLink` crea enlaces que navegan sin recargar la página (un `href` normal sí recargaría toda la app). `routerLinkActive` añade una clase CSS al enlace cuya ruta está activa, útil para resaltar el menú.

```
@Component({
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
      <a routerLink="/" routerLinkActive="activo" [routerLinkActiveOptions]="{
exact: true }">Inicio</a>
      <a routerLink="/productos" routerLinkActive="activo">Productos</a>
      <a [routerLink]="['/productos', producto.id]">Ver producto</a>
    </nav>
    <router-outlet />
  `,
})
export class App {}
```

Navegación programática

A veces hay que navegar **desde código**, por ejemplo después de guardar un formulario o al cerrar sesión. Para eso se inyecta el servicio `Router` y se usa `navigate` (con un array de segmentos) o `navigateByUrl` (con la URL completa).

```
export class FormularioProducto {
  private router = inject(Router);

  async guardar() {
    const nuevo = await this.api.crear(this.datos);
    this.router.navigate(['/productos', nuevo.id], { queryParams: { creado: true }
});
  }
}
```

Guards

Un guard es una función que **decide si se puede entrar (o salir) de una ruta**. Devuelve `true` para permitir, `false` para bloquear, o un `UrlTree` para redirigir. Se usan para proteger zonas privadas, comprobar permisos

o avisar de cambios sin guardar.

- `canActivate`: ¿se puede entrar?
- `canMatch`: ¿esta ruta aplica? (si no, ni siquiera se descarga su código)
- `canDeactivate`: ¿se puede salir?

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(Auth);
  const router = inject(Router);
  return auth.estaLogueado() ? true : router.createUrlTree(['/login']);
};

// en las rutas
{ path: 'perfil', component: Perfil, canActivate: [authGuard] }
```

Resolvers

Un resolver **carga datos antes de activar la ruta**, de forma que el componente nace con los datos ya disponibles. La navegación espera a que el resolver termine. Útil cuando el componente no tiene sentido sin esos datos; para pantallas donde prefieres mostrar un "cargando", es mejor cargar dentro del componente (ver `httpResource` en el capítulo 14).

```
export const productoResolver: ResolveFn<Producto> = (route) =>
  inject(ProductosApi).obtener(route.paramMap.get('id')!);

// ruta
{ path: 'productos/:id', component: DetalleProducto, resolve: { producto:
productoResolver } }

// componente (gracias a withComponentInputBinding)
export class DetalleProducto {
  producto = input.required<Producto>();
}
```

Prácticas del capítulo 10

Práctica 1 · Básico — Web de tres páginas Crea tres componentes (`inicio`, `catalogo`, `contacto`) y define sus rutas con un `title` para cada una. Añade un menú con `routerLink`, resalta la página activa con `routerLinkActive` y crea una página 404 para cualquier ruta no definida.

Práctica 2 · Intermedio — Detalle con parámetros Añade la ruta `catalogo/:id`. Activa `withComponentInputBinding()` y recibe el `id` como `input()` en el componente de detalle. Desde el catálogo, navega al detalle con un enlace y añade en el detalle un botón "Volver" que use `Router.navigate`.

Práctica 3 · Reto — Zona privada Crea un servicio `Auth` con un signal `logueado` y botones para entrar y salir. Protege una ruta `perfil` con un guard que redirija a `/login` si no hay sesión. Carga el perfil con `loadComponent` y comprueba en F12 → Red que su código solo se descarga al entrar en la ruta.

11. Formularios

Los formularios recogen y validan datos del usuario. Angular ofrece varios enfoques; este capítulo se centra en los formularios reactivos y presenta los signal forms.

Enfoques de formularios

Angular ofrece tres maneras de trabajar con formularios:

- **Template-driven** (`ngModel`): la lógica vive en la plantilla. Rápidos para formularios muy simples, pero difíciles de testear y escalar.
- **Reactivos** (`FormGroup`, `FormControl`): el formulario se define en la clase, con tipado y validación explícitos. Son los más usados en proyectos profesionales y existentes.
- **Signal forms**: el modelo del formulario es un signal. Son la apuesta de futuro, pero aún evolucionan.

Recomendación: domina primero los reactivos y después conoce los signal forms.

```
<!-- Template-driven: todo en la plantilla -->
<input [(ngModel)]="email" name="email" required email />
```

Formularios reactivos

Defines la estructura del formulario en la clase: un `FormGroup` agrupa `FormControl`s, cada uno con su valor inicial y sus validadores. `FormBuilder` simplifica la sintaxis, y con `.nonNullable` los valores nunca serán `null` (al resetear vuelven al valor inicial). El formulario está **totalmente tipado**: `getRawValue()` devuelve un objeto con los tipos correctos.

En la plantilla se conecta con `[formGroup]` y `formControlName`, y el envío con `(ngSubmit)`.

```
@Component({
  selector: 'app-registro',
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="enviar()">
      <input formControlName="nombre" placeholder="Nombre" />
      <input formControlName="email" type="email" placeholder="Email" />
      <input formControlName="edad" type="number" />
      <button [disabled]="form.invalid">Registrarme</button>
    </form>
  `,
})
export class Registro {
  private fb = inject(FormBuilder).nonNullable;

  form = this.fb.group({
    nombre: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]],
    edad: [18, [Validators.min(18)]],
  });
```

```
  enviar() {
    if (this.form.invalid) return;
    const datos = this.form.getRawValue(); // { nombre: string; email: string;
edad: number }
    console.log(datos);
  }
}
```

Validación y mensajes de error

Cada control sabe si es válido y qué errores tiene. Para mostrar mensajes se combinan dos cosas: el **error** concreto (`hasError('required')`) y el **estado de interacción** (`touched`: el usuario ya pasó por el campo; `dirty`: lo modificó). Así no se muestran errores antes de que el usuario haya escrito nada.

Validadores integrados: `required`, `email`, `min`, `max`, `minLength`, `maxLength`, `pattern`.

```
<input formControlName="email" />
@if (form.controls.email.touched) {
  @if (form.controls.email.hasError('required')) {
    <small>El email es obligatorio</small>
  } @else if (form.controls.email.hasError('email')) {
    <small>El email no es válido</small>
  }
}
```

Validadores personalizados

Un validador es una **función que recibe el control y devuelve `null` si es válido** o un objeto con el error si no lo es. La clave del objeto es el nombre del error, que luego usas en `hasError()`. Para validar varios campos juntos (por ejemplo, que dos contraseñas coincidan) se aplica el validador al `FormGroup`.

```
export function sinEspacios(control: AbstractControl): ValidationErrors | null {
  return control.value?.includes(' ') ? { sinEspacios: true } : null;
}

export function contrasenasIguales(grupo: AbstractControl): ValidationErrors |
null {
  const { password, repetir } = grupo.value;
  return password === repetir ? null : { noCoinciden: true };
}

form = this.fb.group(
  {
    usuario: ['', [Validators.required, sinEspacios]],
    password: ['', Validators.required],
    repetir: ['', Validators.required],
  },
```

```
    { validators: contrasenasIguales }
  );
```

Signal forms

En los signal forms, el **modelo de datos es un signal** y el formulario se construye a partir de él con `form()`. La validación se declara en una función de esquema que recibe la "ruta" a cada campo. Los cambios del formulario se reflejan en el signal y viceversa, sin `FormGroup` ni suscripciones. Es una API en evolución: consulta la documentación de tu versión antes de usarla en producción.

```
import { form, required, email } from '@angular/forms/signals';

export class Login {
  modelo = signal({ email: '', password: '' });

  loginForm = form(this.modelo, (ruta) => {
    required(ruta.email);
    email(ruta.email);
    required(ruta.password);
  });
}
```

Prácticas del capítulo 11

Práctica 1 · Básico — Formulario de contacto Crea un formulario reactivo con `FormBuilder` que tenga nombre (obligatorio), email (obligatorio y válido) y mensaje (mínimo 20 caracteres). Muestra los errores solo cuando el campo esté `touched` y desactiva el botón de enviar mientras el formulario no sea válido.

Práctica 2 · Intermedio — Registro con validadores propios Crea un formulario de registro con usuario, contraseña y repetir contraseña. Escribe un validador que impida espacios en el usuario y otro, a nivel de grupo, que compruebe que las dos contraseñas coinciden. Al enviar, muestra los datos con `getRawValue()` y resetea el formulario.

Práctica 3 · Reto — Formulario dinámico Crea un formulario de "Pedido" con un `FormArray` de líneas (producto y cantidad) y botones para añadir y quitar líneas. Muestra el total de unidades en tiempo real. Después, reescribe el formulario de contacto de la práctica 1 con signal forms y compara las dos versiones.

---

12. Comunicación con el servidor

Casi todas las aplicaciones obtienen y envían datos a una API. Este capítulo explica qué son los Observables (la forma en que Angular entrega los datos que llegan del servidor), cómo hacer peticiones HTTP con `HttpClient` y cómo aplicar lógica común a todas ellas.

Qué es un Observable

`HttpClient`, el servicio de Angular para hacer peticiones, no devuelve Promises sino **Observables**, así que antes de usarlo hay que entender qué son.

Un **Observable** es, como una Promise, una forma de trabajar con datos que llegan en el futuro. La diferencia es que un Observable puede **emitir muchos valores a lo largo del tiempo**, no solo uno. Es como suscribirse a un canal de YouTube: te suscribes una vez y te avisan cada vez que se publica un vídeo nuevo, hasta que el canal cierra o tú te das de baja.

En la vida de un Observable pueden pasar tres cosas:

Qué pasa	Significado	¿Cuántas veces?
Emite un valor (*next*)	Llega un dato nuevo	Cero, una o muchas
Da un error (*error*)	Algo ha fallado y el Observable se detiene	Como mucho una, y es el final
Se completa (*complete*)	Ya no va a emitir más valores	Como mucho una, y es el final

Algunos ejemplos de Observables en Angular:

- Una **petición HTTP**: emite un único valor (la respuesta) y se completa. O da error si el servidor falla.
- Los **cambios de un campo de formulario** (`valueChanges`): emite un valor cada vez que el usuario escribe y nunca se completa.
- Un **temporizador** con `interval(1000)`: emite un número cada segundo, indefinidamente.

Los Observables vienen de **RxJS**, una librería que Angular instala en todos los proyectos. Por convención, las variables que guardan un Observable terminan en `$` (`pokemon$`, `clics$`), y su tipo se escribe `Observable<Tipo>`, igual que una Promise se escribe `Promise<Tipo>`:

```
import { Observable } from 'rxjs';

// Un Observable que, cuando alguien se suscriba, emitirá un objeto PokemonRespuesta
pokemon$: Observable<PokemonRespuesta> = this.http.get<PokemonRespuesta>(url);
```

Por qué hay que suscribirse

Esta es la diferencia más importante con las Promises. Un Observable es **perezoso**: crearlo **no ejecuta nada**. Es solo la descripción de lo que hay que hacer, como una receta escrita que nadie ha cocinado todavía. El trabajo empieza en el momento en que alguien **se suscribe** con `subscribe()`.

En el caso de `HttpClient`, eso significa que **la petición no sale hacia el servidor hasta que te suscribes**. Si llamas al método de un servicio pero no te suscribes, no pasa nada: ni se pide nada ni da error.

```
// ❌ No hace ninguna petición: solo prepara el Observable
this.pokemonService.getPokemon('pikachu');

// ✅ Ahora sí se envía la petición y, cuando llega, se ejecuta la función
this.pokemonService.getPokemon('pikachu').subscribe((datos) => {
  console.log(datos.name);
});
```

Puedes comprobarlo en F12 → Red: sin `subscribe` no aparece ninguna petición.

Dos consecuencias prácticas:

- **Cada `subscribe` vuelve a ejecutar el Observable.** Si te suscribes dos veces al mismo Observable de `HttpClient`, se hacen **dos peticiones**. No te suscribas más veces de las necesarias.
- Una Promise, en cambio, **empieza a ejecutarse al crearla**, haya o no alguien esperando el resultado.

Opciones del método subscribe

`subscribe()` recibe lo que hay que hacer en cada uno de los tres casos: cuando llega un valor, cuando hay un error y cuando se completa. Hay dos formas de escribirlo.

1. Pasando solo una función. Es la forma corta, y la función se ejecuta con cada valor que llegue. Sirve cuando no necesitas gestionar errores (por ejemplo, en una prueba rápida):

```
this.pokemonService.getPokemon('pikachu').subscribe((datos) => {
  this.pokemon.set(datos);
});
```

2. Pasando un objeto con `next`, `error` y `complete`. Es la forma completa y la recomendada cuando la operación puede fallar, como cualquier petición HTTP. Las tres propiedades son opcionales: pon solo las que necesites.

Propiedad	Cuándo se ejecuta	Qué recibe
`next`	Cada vez que el Observable emite un valor	El valor emitido
`error`	Si el Observable falla. Después ya no se ejecuta nada más	El error
`complete`	Cuando el Observable termina correctamente. No se ejecuta si hubo error	Nada

```
this.cargando.set(true);

this.pokemonService.getPokemon(nombre).subscribe({
  next: (datos) => {
    this.pokemon.set(datos);
  },
  error: (err: HttpErrorResponse) => {
    if (err.status === 404) {
      this.error.set(`El Pokémon "${nombre}" no existe`);
    } else {
      this.error.set('No se ha podido conectar con el servidor');
    }
    this.cargando.set(false);
  },
  complete: () => {
    this.cargando.set(false);
```

```
  },
});
```

Fíjate en que `cargando.set(false)` está en `error` y en `complete`, porque solo se ejecuta uno de los dos. Para no repetirlo, puedes usar el operador `finalize`, que se ejecuta siempre al terminar, igual que el `finally` de las Promises:

```
import { finalize } from 'rxjs';

this.pokemonService.getPokemon(nombre)
  .pipe(finalize(() => this.cargando.set(false)))
  .subscribe({
    next: (datos) => this.pokemon.set(datos),
    error: () => this.error.set('No se pudo cargar'),
  });
```

A diferencia de `fetch`, **en `HttpClient` un 404 o un 500 sí son errores**: cualquier código de estado fuera del rango 200–299 llega a la función `error` como un `HttpErrorResponse`, que tiene las propiedades `status` (el código) y `message`.

Si un Observable da error y no has indicado la función `error`, el error aparece en la consola del navegador como "no gestionado". En las peticiones HTTP, pon siempre `error`.

Cancelar una suscripción

`subscribe()` devuelve un objeto `Subscription`. Llamando a su método `unsubscribe()` te das de baja: dejas de recibir valores y, si era una petición HTTP en curso, **se cancela**.

```
const suscripcion = this.pokemonService.getPokemon('pikachu').subscribe((datos) =>
{
  console.log(datos.name);
});

suscripcion.unsubscribe(); // cancela la petición si aún no había llegado
```

¿Cuándo hace falta darse de baja?

- **Peticiones HTTP**: normalmente no. Emiten un valor y se completan solas, y al completarse la suscripción termina.
- **Observables que no terminan nunca** (`interval`, `valueChanges`, websockets): sí. Si no, siguen funcionando aunque el componente haya desaparecido, gastando memoria. En Angular se hace de forma automática con `takeUntilDestroyed()`, que se explica en el capítulo 13.

Observable frente a Promise

	Promise	**Observable**
Cuántos valores	Uno	Cero, uno o muchos
Cuándo empieza a ejecutarse	Al crearla	Al suscribirse
Cómo se recibe el valor	`await` o `.then()`	`.subscribe()`
Gestión de errores	`try/catch` o `.catch()`	Función `error` del `subscribe`
Se puede cancelar	No	Sí, con `unsubscribe()`
Transformar los datos	A mano	Con operadores de RxJS (`map`, `filter`...)
Dónde se usa	`fetch` y JavaScript en general	`HttpClient`, formularios y router de Angular

Si prefieres trabajar con `async/await`, puedes convertir un Observable en Promise con `firstValueFrom`, que espera al primer valor emitido:

```
import { firstValueFrom } from 'rxjs';

async cargar() {
  try {
    const datos = await firstValueFrom(this.pokemonService.getPokemon('pikachu'));
    this.pokemon.set(datos);
  } catch (err) {
    this.error.set('No se pudo cargar');
  }
}
```

Las dos formas son válidas. En Angular lo habitual es `subscribe`, porque los Observables permiten cancelar y encadenar operaciones (capítulo 13), y porque Angular tiene herramientas como `toSignal` para convertirlos directamente en signals (capítulo 14).

Configurar HttpClient

`HttpClient` es el servicio de Angular para **hacer peticiones HTTP** (GET, POST, PUT, DELETE...). Se registra en los providers con `provideHttpClient`, donde también se añaden los interceptores. Convierte automáticamente las respuestas JSON a objetos.

```
// app.config.ts
providers: [
  provideHttpClient(withInterceptors([authInterceptor, errorInterceptor])),
]
```

Servicio de API

Lo recomendable es **agrupar las llamadas a cada recurso en un servicio**, en lugar de usar `HttpClient` directamente en los componentes. Cada método devuelve un Observable tipado con el genérico (`get<Producto[]>`). Importante: la petición **no se envía hasta que alguien se suscribe** (o la convierte en signal).

`http` y `base` son `private` porque son detalles internos: los componentes solo necesitan llamar a los métodos, no saber qué URL se usa ni cómo se hace la petición.

```
import { Service, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Service()
export class ProductosApi {
  private readonly http = inject(HttpClient);
  private readonly base = '/api/productos';

  listar()                        { return this.http.get<Producto[]>(this.base);
}
  obtener(id: string)             { return this.http.get<Producto>
(`${this.base}/${id}`); }
  buscar(termino: string)         { return this.http.get<Producto[]>(this.base, {
params: { q: termino } }); }
  crear(p: Omit<Producto, 'id'>)  { return this.http.post<Producto>(this.base,
p); }
  actualizar(id: number, p: Partial<Producto>) { return this.http.patch<Producto>
(`${this.base}/${id}`, p); }
  borrar(id: number)              { return this.http.delete<void>
(`${this.base}/${id}`); }
}
```

Interceptores

Un interceptor es una función por la que **pasan todas las peticiones y respuestas HTTP**. Permite aplicar lógica común en un único sitio: añadir el token de autenticación, gestionar errores globales, mostrar un indicador de carga, registrar tiempos. Las peticiones son inmutables, así que para modificarlas se usa `req.clone()`. `next(req)` pasa la petición al siguiente interceptor (o al servidor).

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(Auth).token();
  if (!token) return next(req);
  return next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
};

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) router.navigate(['/login']);
      return throwError(() => err);
```

```
    })
  );
};
```

Prácticas del capítulo 12

Práctica 1 · Básico — Buscar un Pokémon Crea un servicio `PokemonService` con un método `getPokemon(nombre)` que use `HttpClient` y una interfaz `PokemonRespuesta` con `id`, `name` y `sprites.front_default`. En un componente, añade un input para buscar y muestra el nombre, el número y la imagen.

Práctica 2 · Intermedio — Gestión de errores Amplía la práctica anterior: muestra "Cargando..." mientras llega la respuesta, un mensaje distinto si el Pokémon no existe (404) o si no hay conexión, y limpia el resultado anterior al empezar una búsqueda nueva. Pasa siempre el nombre a minúsculas y sin espacios.

Práctica 3 · Reto — Interceptor Crea un interceptor que registre en consola la URL y el tiempo que tarda cada petición, y otro que muestre un indicador global de "cargando" mientras haya alguna petición en curso (usa un servicio con un contador en un signal). Regístralos en `app.config.ts` con `withInterceptors`.

Repaso de la Parte II

- [ ] Gestiono estado con `signal` y `computed`, y uso `effect` solo para efectos
- [ ] Actualizo arrays y objetos de forma inmutable
- [ ] Comunico componentes con `input`, `output` y `model`
- [ ] Encapsulo estado compartido en servicios con signals privados
- [ ] Configuro rutas con lazy loading, guards, resolvers y parámetros como inputs
- [ ] Construyo formularios reactivos tipados con validación propia
- [ ] Entiendo qué es un Observable y uso `subscribe` con `next`, `error` y `complete`
- [ ] Consumo una API REST con interceptores

---

13. Programación reactiva con RxJS

RxJS es la librería con la que Angular gestiona flujos de eventos asíncronos: teclas, peticiones, temporizadores o websockets. Dominar sus operadores principales es imprescindible en proyectos reales.

RxJS y pipe()

Qué es un Observable, por qué hay que suscribirse y qué opciones tiene `subscribe` se explica en el capítulo 12. Este capítulo se centra en lo que hace potente a RxJS: los **operadores**, funciones que transforman y combinan Observables. Se encadenan dentro de `pipe()`, y cada operador recibe los valores del anterior y entrega los suyos al siguiente.

```
import { interval, map, take } from 'rxjs';

const segundos$ = interval(1000).pipe(   // emite 0, 1, 2… cada segundo
  map(n => n + 1),
  take(5),                               // se completa tras 5 valores
);
```

```
segundos$.subscribe({
  next: v => console.log(v),              // 1, 2, 3, 4, 5
  complete: () => console.log('Fin'),
});
```

Operadores de transformación

- `map`: transforma cada valor.
- `filter`: deja pasar solo los valores que cumplen una condición.
- `tap`: ejecuta un efecto secundario (log, depuración) **sin alterar** el valor.

```
this.api.listar().pipe(
  tap(lista => console.log('Recibidos', lista.length)),
  map(lista => lista.filter(p => p.stock > 0)),
);

clicks$.pipe(filter(e => e.shiftKey));   // solo clics con Shift pulsado
```

Control de eventos rápidos

- `debounceTime(ms)`: espera a que pasen X milisegundos **sin nuevos valores** antes de emitir el último. Evita lanzar una petición por cada tecla.
- `distinctUntilChanged()`: ignora un valor si es **igual al anterior**.

Juntos son el patrón clásico de un buscador.

```
this.termino.valueChanges.pipe(
  debounceTime(300),          // el usuario deja de escribir 300 ms
  distinctUntilChanged(),     // y el texto ha cambiado realmente
);
```

Operadores de aplanamiento

Cuando por cada valor de un Observable necesitas lanzar **otro Observable** (por ejemplo, una petición HTTP por cada búsqueda), usas un operador de aplanamiento. Elegir bien es crucial, porque cada uno gestiona de forma distinta las peticiones que se solapan:

Operador	Comportamiento	Caso de uso
`switchMap`	Cancela la petición anterior al llegar una nueva	Buscadores, cambios de filtro
`mergeMap`	Ejecuta todas en paralelo	Subir varios archivos a la vez
`concatMap`	Ejecuta en orden, una tras otra	Guardados que deben respetar el orden
`exhaustMap`	Ignora nuevas mientras hay una en curso	Botón "Guardar" con doble clic

```
// Buscador: solo importa la última búsqueda
termino$.pipe(switchMap(t => this.api.buscar(t)));

// Botón de guardar: ignora clics repetidos mientras se guarda
guardar$.pipe(exhaustMap(() => this.api.crear(this.form.getRawValue())));
```

Manejo de errores

Si un Observable emite un error, **se detiene** y no emite más valores. `catchError` captura el error y devuelve un Observable alternativo (por ejemplo, una lista vacía) para que el flujo continúe. `retry` vuelve a intentarlo un número de veces, opcionalmente con espera entre intentos.

```
this.api.listar().pipe(
  retry({ count: 2, delay: 1000 }),    // reintenta 2 veces, esperando 1 s
  catchError(() => of([])),            // si sigue fallando, lista vacía
);
```

Ojo: dentro de un `switchMap`, pon el `catchError` en el Observable interno; si lo pones fuera, el primer error mata todo el flujo del buscador.

Combinar flujos

- `forkJoin`: espera a que **todos terminen** y emite una vez con todos los resultados. Ideal para cargar varios recursos en paralelo al abrir una pantalla.
- `combineLatest`: emite **cada vez que cualquiera cambia**, con el último valor de todos. Útil para combinar filtros.

```
// Cargar todo lo necesario para un panel
forkJoin({
  usuario: this.usuariosApi.obtener(id),
  pedidos: this.pedidosApi.listarDe(id),
}).subscribe(({ usuario, pedidos }) => { /* ... */ });

// Recalcular cuando cambia cualquier filtro
combineLatest([categoria$, orden$]).pipe(
  switchMap(([cat, ord]) => this.api.filtrar(cat, ord)),
);
```

Evitar fugas de memoria

Una suscripción que nunca se cancela sigue viva aunque el componente se haya destruido: consume memoria y puede ejecutar código sobre una vista que ya no existe. Las peticiones HTTP se completan solas, pero los Observables infinitos (`interval`, `valueChanges`, websockets) no. `takeUntilDestroyed()` cancela la suscripción automáticamente cuando el componente se destruye. Mejor aún: convierte el Observable en signal con `toSignal`, que también se limpia solo.

```
export class Reloj {
  constructor() {
    interval(1000)
      .pipe(takeUntilDestroyed())
      .subscribe(n => console.log(n));
  }
}
```

Prácticas del capítulo 13

Práctica 1 · Básico — Primeros Observables Crea un Observable con `interval(1000)` que emita los segundos, transfórmalo con `map` para mostrar "Han pasado X segundos" y detenlo a los 10 con `take`. Muestra el valor en pantalla con `toSignal` y comprueba que deja de emitir al destruir el componente.

Práctica 2 · Intermedio — Buscador con debounce Crea un buscador de Pokémon que busque mientras el usuario escribe, usando `debounceTime(400)`, `distinctUntilChanged`, `filter` (mínimo 3 letras) y `switchMap`. Gestiona el 404 con `catchError` para que un nombre inexistente no detenga el buscador.

Práctica 3 · Reto — Combinar peticiones Crea una pantalla que, al elegir un Pokémon, cargue en paralelo con `forkJoin` sus datos (`/pokemon/{id}`) y su especie (`/pokemon-species/{id}`) y muestre la imagen junto con la descripción en español. Añade un botón "Guardar favorito" que use `exhaustMap` para ignorar los clics repetidos mientras se guarda (simula el guardado con `timer(1000)`).

---

14. Carga de datos asíncrona

Este capítulo explica cómo convivir con signals y RxJS a la vez y presenta la API de recursos, que simplifica la carga de datos con sus estados de carga y error.

Signals frente a RxJS

No compiten: cada uno tiene su terreno.

- **Signals → estado**: lo que la vista muestra en un momento dado.
- **RxJS → eventos y flujos asíncronos complejos**: debounce, cancelaciones, reintentos, websockets.

Para pasar de uno a otro existen `toSignal(observable$)` (para usar el resultado en la plantilla) y `toObservable(signal)` (para aplicar operadores de RxJS a un signal).

```
export class Buscador {
  private api = inject(ProductosApi);
  termino = new FormControl('', { nonNullable: true });

  resultados = toSignal(
    this.termino.valueChanges.pipe(
      debounceTime(300),
      distinctUntilChanged(),
      filter(t => t.length >= 2),
      switchMap(t => this.api.buscar(t).pipe(catchError(() => of([])))),
```

```
    ),
    { initialValue: [] }
  );
}
```

```
@for (p of resultados(); track p.id) { <p>{{ p.nombre }}</p> }
```

httpResource

`httpResource` hace una petición GET y expone el resultado como **signals de estado**: `value()`, `isLoading()`, `error()` y `hasValue()`. Su gran ventaja es que **se recarga sola cuando cambian los signals que usa en la URL**: si cambia el `id`, vuelve a pedir los datos y cancela la petición anterior. Elimina mucho código repetitivo de carga, error y suscripción.

```
export class DetalleUsuario {
  id = input.required<number>();
  usuario = httpResource<Usuario>(() => `/api/usuarios/${this.id()}`);
}
```

```
@if (usuario.isLoading()) {
  <p>Cargando…</p>
} @else if (usuario.error()) {
  <p>Error al cargar</p>
} @else if (usuario.hasValue()) {
  <h2>{{ usuario.value().nombre }}</h2>
}
<button (click)="usuario.reload()">Recargar</button>
```

resource

Es la versión **genérica** de `httpResource`: funciona con cualquier función que devuelva una Promise (un `fetch`, una librería externa, IndexedDB). `params` define de qué signals depende; cuando cambian, se ejecuta `loader` de nuevo. `abortSignal` permite cancelar la operación anterior.

```
export class Clima {
  ciudad = signal('Bilbao');

  clima = resource({
    params: () => ({ ciudad: this.ciudad() }),
    loader: ({ params, abortSignal }) =>
      fetch(`/api/clima?c=${params.ciudad}`, { signal: abortSignal }).then(r =>
r.json()),
```

```
  });
}
```

Prácticas del capítulo 14

Práctica 1 · Básico — httpResource Crea un componente con un signal `id` y botones "Anterior" y "Siguiente". Usa `httpResource` para cargar `https://pokeapi.co/api/v2/pokemon/{id}` y muestra los estados de carga, error y datos con `isLoading()`, `error()` y `hasValue()`.

Práctica 2 · Intermedio — resource con fetch Reescribe la práctica anterior con `resource` y `fetch` en lugar de `httpResource`, usando `abortSignal` para cancelar la petición anterior. Pulsa "Siguiente" muy rápido varias veces y comprueba en F12 → Red que las peticiones antiguas se cancelan.

Práctica 3 · Reto — De RxJS a signals y viceversa Partiendo del buscador con debounce del capítulo 13, guarda el término de búsqueda en un signal, conviértelo en Observable con `toObservable`, aplica los operadores y vuelve a convertir el resultado en signal con `toSignal`. Explica en un comentario qué parte resuelve mejor cada herramienta.

---

15. Detección de cambios

La detección de cambios es el mecanismo por el que Angular decide cuándo y qué actualizar en la vista. Entenderlo explica muchos errores habituales y es la clave del rendimiento.

Change detection

Es el proceso por el que Angular **comprueba si el estado ha cambiado y actualiza la vista** en consecuencia. Entender cuándo se ejecuta explica por qué a veces la pantalla "no se actualiza" y cómo hacer apps más rápidas. Angular lanza una comprobación cuando:

- cambia un signal leído en una plantilla,
- se dispara un evento enlazado en la plantilla (`(click)`…),
- cambia un input del componente,
- emite un Observable usado con el pipe `async`.

```
export class Ejemplo {
  contador = signal(0);       // al cambiar, Angular actualiza la vista
  normal = 0;

  constructor() {
    setTimeout(() => this.contador.set(1), 1000); // ✅ se pinta
    setTimeout(() => this.normal = 1, 1000);      // ❌ en zoneless no se pinta
  }
}
```

Zone.js y zoneless

Antes, Angular usaba **Zone.js**, una librería que "parcheaba" todos los eventos asíncronos del navegador (clics, timers, peticiones) para lanzar una revisión de **toda la aplicación** tras cada uno. Funcionaba sin que el desarrollador pensara en ello, pero era costoso y poco predecible.

Las aplicaciones nuevas son **zoneless**: Angular ya no necesita Zone.js porque los signals le dicen exactamente qué ha cambiado. Resultado: apps más ligeras y rápidas, y comportamiento más predecible. En proyectos antiguos se puede activar manualmente.

```
// app.config.ts (en proyectos antiguos que quieras migrar)
providers: [provideZonelessChangeDetection()]
```

OnPush

Con la estrategia `OnPush`, un componente **solo se revisa** cuando cambia uno de sus inputs (por referencia), se dispara un evento en él o cambia un signal que lee. Así Angular se salta componentes enteros que no han cambiado. Úsalo **por defecto** en todos los componentes; combinado con signals no requiere esfuerzo extra.

```
@Component({
  selector: 'app-lista',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `@for (t of tareas(); track t.id) { <li>{{ t.titulo }}</li> }`,
})
export class Lista {
  tareas = input.required<Tarea[]>();
}
```

Errores típicos con zoneless y OnPush

Casi todos los problemas de "la vista no se actualiza" vienen de lo mismo: **cambiar datos de una forma que Angular no puede detectar**.

- Mutar un objeto o array en lugar de crear uno nuevo.
- Cambiar una propiedad normal (no signal) dentro de un `setTimeout`, una promesa o una librería externa.

La solución en ambos casos: guardar el estado en signals y actualizarlos de forma inmutable.

```
// ❌ La vista no se entera
this.usuario.nombre = 'Ana';
fetch(url).then(r => r.json()).then(d => this.datos = d);

// ✅ Correcto
this.usuario.update(u => ({ ...u, nombre: 'Ana' }));
fetch(url).then(r => r.json()).then(d => this.datos.set(d));
```

Prácticas del capítulo 15

Práctica 1 · Básico — Encontrar el fallo Crea un componente con una propiedad normal `mensaje = 'Hola'` y cámbiala dentro de un `setTimeout` a los 2 segundos. Comprueba que la vista no se actualiza en una app zoneless y arréglalo con un signal.

Práctica 2 · Intermedio — Mutar frente a crear Crea una lista de usuarios en un signal y un componente hijo con `OnPush` que la reciba con `input()`. Añade un usuario con `push` sobre el array y observa que el hijo no se actualiza; después hazlo de forma inmutable y comprueba la diferencia.

Práctica 3 · Reto — Medir la detección de cambios Instala Angular DevTools en el navegador y usa la pestaña Profiler para medir una lista de 500 elementos en la que cambias un único elemento. Compara los resultados con y sin `OnPush` en los componentes de la lista y anota cuántos componentes se revisan en cada caso.

---

16. Angular Material y CDK

Angular Material es la librería oficial de componentes visuales, y el CDK es la base de comportamientos sobre la que está construida. Juntas evitan reinventar piezas complejas de interfaz.

Angular Material

Es la **librería oficial de componentes visuales** de Angular, basada en Material Design: botones, formularios, tablas, diálogos, menús, pestañas, etc. Son accesibles, están probados y siguen un diseño coherente. Se instala con `ng add`, que configura tema, fuentes e iconos. Cada componente se importa por separado.

```
ng add @angular/material
```

```
import { MatButtonModule } from '@angular/material/button';
import { MatDialog } from '@angular/material/dialog';

@Component({
  imports: [MatButtonModule],
  template: `<button mat-flat-button (click)="abrir()">Confirmar</button>`,
})
export class Acciones {
  private dialog = inject(MatDialog);

  abrir() {
    const ref = this.dialog.open(DialogoConfirmacion, { data: { mensaje:
'¿Seguro?' } });
    ref.afterClosed().subscribe(confirmado => { if (confirmado) this.borrar(); });
  }
}
```

Angular CDK

El CDK (Component Dev Kit) ofrece **comportamientos sin estilos**: la lógica de un componente complejo sin imponer un diseño. Es la base sobre la que está construido Material, y lo puedes usar con tu propio diseño. Incluye drag & drop, overlays (popups posicionados), virtual scroll, gestión de foco y utilidades de accesibilidad.

```
import { CdkDropList, CdkDrag, CdkDragDrop, moveItemInArray } from
'@angular/cdk/drag-drop';

@Component({
  imports: [CdkDropList, CdkDrag],
  template: `
    <ul cdkDropList (cdkDropListDropped)="soltar($event)">
      @for (t of tareas(); track t.id) { <li cdkDrag>{{ t.titulo }}</li> }
    </ul>
  `,
})
export class Tablero {
  tareas = signal<Tarea[]>([]);

  soltar(e: CdkDragDrop<Tarea[]>) {
    this.tareas.update(l => {
      const copia = [...l];
      moveItemInArray(copia, e.previousIndex, e.currentIndex);
      return copia;
    });
  }
}
```

Prácticas del capítulo 16

Práctica 1 · Básico — Instalar Material Añade Angular Material al proyecto con `ng add @angular/material` (instala la librería y configura el tema, las fuentes y los iconos). Sustituye los botones de una de tus prácticas por botones de Material y añade una barra superior con `mat-toolbar`.

Práctica 2 · Intermedio — Diálogo de confirmación En la lista de tareas, al pulsar "Eliminar" abre un diálogo de Material que pregunte "¿Seguro que quieres borrar esta tarea?" con el título de la tarea. Elimínala solo si el usuario confirma, leyendo el resultado con `afterClosed()`.

Práctica 3 · Reto — Tablero Kanban con el CDK Crea un tablero con tres columnas (Pendiente, En curso, Hecho) usando drag & drop del CDK, de forma que las tareas se puedan reordenar dentro de una columna y mover entre columnas. Guarda el estado en signals y actualízalo de forma inmutable.

---

17. Pruebas

Las pruebas automáticas garantizan que la aplicación funciona y sigue funcionando cuando cambia. Este capítulo cubre pruebas unitarias de servicios y componentes, peticiones HTTP simuladas y pruebas end-to-end.

Testing con Vitest y TestBed

Los tests automáticos comprueban que el código hace lo que debe y que **sigue haciéndolo cuando cambias algo**. Los proyectos nuevos usan **Vitest** como ejecutor de tests (se lanzan con `ng test`). `TestBed` es la utilidad de Angular que crea un entorno de pruebas con inyección de dependencias, donde puedes crear componentes y servicios y sustituir dependencias por versiones falsas.

Estructura de un test: `describe` agrupa, `beforeEach` prepara, `it` define un caso y `expect` comprueba.

```
describe('Calculadora', () => {
  it('suma dos números', () => {
    expect(sumar(2, 3)).toBe(5);
  });
});
```

Test de un servicio

Es el tipo de test más sencillo y valioso: comprueba la lógica de negocio sin interfaz. Se obtiene el servicio con `TestBed.inject` y se verifican sus métodos y signals.

```
describe('Carrito', () => {
  let carrito: Carrito;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    carrito = TestBed.inject(Carrito);
  });

  it('calcula el total', () => {
    carrito.anadir({ id: 1, precio: 10, cantidad: 2 });
    carrito.anadir({ id: 2, precio: 5, cantidad: 1 });
    expect(carrito.total()).toBe(25);
  });

  it('vacía el carrito', () => {
    carrito.anadir({ id: 1, precio: 10, cantidad: 1 });
    carrito.vaciar();
    expect(carrito.total()).toBe(0);
  });
});
```

Test de un componente

Se crea el componente con `TestBed.createComponent`, se interactúa con su DOM como lo haría un usuario y se comprueba lo que se ve. `fixture.whenStable()` espera a que Angular termine de actualizar la vista. Para pasar inputs se usa `fixture.componentRef.setInput()`. Testea **comportamiento visible**, no detalles internos.

```
describe('Contador', () => {
  it('incrementa al pulsar', async () => {
    const fixture = TestBed.createComponent(Contador);
    await fixture.whenStable();

    fixture.nativeElement.querySelector('button').click();
    await fixture.whenStable();

    expect(fixture.nativeElement.textContent).toContain('Contador: 1');
  });

  it('muestra el producto recibido', async () => {
    const fixture = TestBed.createComponent(TarjetaProducto);
    fixture.componentRef.setInput('producto', { id: 1, nombre: 'Silla', precio: 40
});
    await fixture.whenStable();
    expect(fixture.nativeElement.textContent).toContain('Silla');
  });
});
```

Test con HTTP simulado

No conviene que los tests llamen a una API real: serían lentos e inestables. `provideHttpClientTesting()` intercepta las peticiones, y `HttpTestingController` permite **comprobar qué petición se hizo y responder con datos falsos** (`flush`). `verify()` asegura que no quedó ninguna petición sin atender.

```
beforeEach(() => {
  TestBed.configureTestingModule({
    providers: [provideHttpClient(), provideHttpClientTesting()],
  });
});

it('lista productos', () => {
  const api = TestBed.inject(ProductosApi);
  const http = TestBed.inject(HttpTestingController);

  let resultado: Producto[] = [];
  api.listar().subscribe(r => (resultado = r));

  http.expectOne('/api/productos').flush([{ id: 1, nombre: 'Silla', precio: 40
}]);
  expect(resultado.length).toBe(1);
  http.verify();
});
```

Tests end-to-end con Playwright

Los tests e2e **controlan un navegador real** y recorren la aplicación como un usuario: abren páginas, hacen clic, rellenan formularios. Son más lentos, pero detectan fallos que los tests unitarios no ven (integración entre partes, rutas, estilos que ocultan botones). Playwright localiza elementos preferentemente por su **rol y texto accesible**, lo que de paso fomenta una interfaz accesible.

```
import { test, expect } from '@playwright/test';

test('el usuario puede añadir al carrito', async ({ page }) => {
  await page.goto('/productos');
  await page.getByRole('button', { name: 'Añadir' }).first().click();
  await expect(page.getByText('Total: 40 €')).toBeVisible();
});
```

Qué testear

No hace falta testearlo todo; prioriza donde un fallo duele más:

- Lógica de servicios y `computed` (cálculos, reglas de negocio).
- Comportamiento visible de los componentes importantes.
- Guards e interceptores (seguridad y errores).
- Los 3–5 flujos críticos de la aplicación en e2e (login, compra, alta…).

```
// Un guard también se testea como una función
it('redirige a /login si no hay sesión', () => {
  TestBed.configureTestingModule({
    providers: [{ provide: Auth, useValue: { estaLogueado: () => false } }],
  });
  const resultado = TestBed.runInInjectionContext(() => authGuard({} as any, {} as
any));
  expect(resultado.toString()).toBe('/login');
});
```

Prácticas del capítulo 17

Práctica 1 · Básico — Testear un servicio Escribe tests para el servicio de favoritos del capítulo 9: que empiece vacío, que `anadir` incremente el total, que no permita duplicados y que `quitar` elimine el elemento correcto. Ejecútalos con `ng test` (lanza todos los tests del proyecto y muestra cuáles pasan).

Práctica 2 · Intermedio — Testear un componente Testea el componente `contador-cantidad` del capítulo 5: que muestre el valor inicial recibido, que el botón + lo incremente, que el botón − no baje de 0 y que no supere el máximo. Usa `fixture.componentRef.setInput()` para pasar los inputs.

Práctica 3 · Reto — HTTP simulado y e2e Testea el `PokemonService` con `HttpTestingController`: comprueba la URL pedida, responde con datos falsos y simula un 404 con `flush(null, { status: 404, statusText: 'Not Found' })`. Después escribe un test con Playwright que busque "pikachu" en la aplicación y compruebe que aparece su imagen.

Repaso de la Parte III

- [ ] Elijo el operador de aplanamiento correcto (`switchMap`, `exhaustMap`...)
- [ ] Gestiono errores y suscripciones sin fugas de memoria
- [ ] Sé cuándo usar signals, RxJS, `httpResource` o `resource`
- [ ] Entiendo change detection, OnPush y zoneless
- [ ] Uso Material y el CDK
- [ ] Escribo tests de servicios, componentes, HTTP y e2e

---

18. Gestión del estado global

Cuando una aplicación crece, el estado compartido entre muchas pantallas necesita una estructura clara. Este capítulo explica cuándo conviene un store y cómo crearlo con NgRx SignalStore.

Cuándo usar un store

Un **store** es un lugar centralizado donde vive el estado de una parte de la aplicación junto con las operaciones que lo modifican. Cuando muchas pantallas leen y modifican los mismos datos, tener el estado repartido en servicios distintos se vuelve difícil de seguir. Un store da una estructura común: estado, valores derivados y métodos.

Cuándo **no** lo necesitas: en apps pequeñas o para estado local de un componente. Un servicio con signals (como el `Carrito` del capítulo 9) suele bastar.

```
// Señal de que necesitas un store: lógica repetida y estado disperso
// ListaTareas, ContadorPendientes, FiltroTareas y EditorTarea
// leen y modifican las mismas tareas desde sitios distintos.
```

NgRx SignalStore

`SignalStore` es la propuesta de NgRx basada en signals. Se construye **componiendo piezas**:

- `withState`: el estado inicial. Cada propiedad se convierte en un signal.
- `withComputed`: valores derivados.
- `withMethods`: operaciones que modifican el estado con `patchState`, incluidas las asíncronas.

`patchState` actualiza solo las propiedades indicadas de forma inmutable.

```
npm install @ngrx/signals
```

```
import { signalStore, withState, withComputed, withMethods, patchState } from
'@ngrx/signals';

type EstadoTareas = { tareas: Tarea[]; filtro: 'todas' | 'pendientes' | 'hechas';
cargando: boolean };
```

```
export const TareasStore = signalStore(
  { providedIn: 'root' },
  withState<EstadoTareas>({ tareas: [], filtro: 'todas', cargando: false }),

  withComputed(({ tareas, filtro }) => ({
    visibles: computed(() => {
      const f = filtro();
      return f === 'todas' ? tareas() : tareas().filter(t => t.hecha === (f ===
'hechas'));
    }),
    pendientes: computed(() => tareas().filter(t => !t.hecha).length),
  })),

  withMethods((store, api = inject(TareasApi)) => ({
    setFiltro(filtro: EstadoTareas['filtro']) {
      patchState(store, { filtro });
    },
    async cargar() {
      patchState(store, { cargando: true });
      const tareas = await firstValueFrom(api.listar());
      patchState(store, { tareas, cargando: false });
    },
    alternar(id: number) {
      patchState(store, s => ({ tareas: s.tareas.map(t => t.id === id ? { ...t,
hecha: !t.hecha } : t) }));
    },
  })),
);
```

Usar el store

El store se inyecta como cualquier servicio. Sus propiedades de estado y computados son signals, y sus métodos se llaman directamente. Los componentes quedan muy simples: leen y llaman a métodos, sin lógica propia.

```
@Component({
  selector: 'app-lista-tareas',
  template: `
    <p>Pendientes: {{ store.pendientes() }}</p>
    <button (click)="store.setFiltro('pendientes')">Ver pendientes</button>

    @if (store.cargando()) { <p>Cargando…</p> }
    @for (t of store.visibles(); track t.id) {
      <li (click)="store.alternar(t.id)">{{ t.titulo }}</li>
    }
  `,
})
export class ListaTareas {
  store = inject(TareasStore);
```

```
  constructor() { this.store.cargar(); }
}
```

Prácticas del capítulo 18

Práctica 1 · Básico — ¿Necesito un store? Revisa la lista de tareas y el servicio de favoritos que has hecho en capítulos anteriores. Para cada uno, razona si merece la pena un store o basta con un servicio con signals, teniendo en cuenta cuántos componentes leen y modifican esos datos.

Práctica 2 · Intermedio — Store de tareas Instala NgRx Signals con `npm install @ngrx/signals` (descarga la librería y la añade a `package.json`). Convierte la lista de tareas en un `signalStore` con `withState`, un `withComputed` para las pendientes y las visibles según un filtro, y `withMethods` para añadir, borrar, alternar y filtrar.

Práctica 3 · Reto — Store con datos de una API Crea un store de Pokémon favoritos que cargue los datos desde la PokeAPI en un método asíncrono, gestione los estados `cargando` y `error` y guarde los favoritos en `localStorage`. Usa el store desde al menos tres componentes: la lista, un contador en la cabecera y el detalle.

---

19. Renderizado en servidor

Renderizar en el servidor mejora la velocidad de la primera carga y el posicionamiento en buscadores. Este capítulo cubre SSR, prerenderizado e hidratación.

SSR (renderizado en servidor)

Normalmente el navegador recibe un HTML vacío y JavaScript pinta todo. Con **SSR** (Server-Side Rendering), el **servidor genera el HTML completo** de cada página antes de enviarlo. Ventajas: el usuario ve contenido antes y los buscadores indexan mejor la página (SEO). Requiere un servidor Node para ejecutar la app.

```
ng add @angular/ssr   # en un proyecto existente
ng new mi-app --ssr   # en un proyecto nuevo
```

SSG o prerenderizado

El **prerenderizado** (SSG, Static Site Generation) genera el HTML de las páginas **en tiempo de compilación**, no en cada petición. Las páginas resultantes son archivos estáticos que se sirven rapidísimo y desde cualquier hosting. Es ideal para contenido que no cambia por usuario: portada, páginas informativas, blog.

```
// app.routes.server.ts
export const serverRoutes: ServerRoute[] = [
  { path: '', renderMode: RenderMode.Prerender },
  { path: 'sobre-nosotros', renderMode: RenderMode.Prerender },
];
```

Hidratación

Cuando el navegador recibe el HTML del servidor, Angular tiene que "tomar el control" para que la página sea interactiva. La **hidratación** consiste en **reutilizar el DOM que ya llegó** en vez de destruirlo y volver a pintarlo. Evita parpadeos y mejora el rendimiento. Viene activada por defecto en proyectos con SSR.

```
// app.config.ts
providers: [provideClientHydration()]
```

Hidratación incremental

Lleva la hidratación más allá: en lugar de hidratar toda la página de golpe, cada bloque `@defer` se hidrata **solo cuando hace falta** (al entrar en pantalla, al interactuar...). Hasta entonces, el usuario ve el HTML del servidor pero no se descarga ni ejecuta su JavaScript. Reduce mucho el trabajo inicial en páginas largas.

```
// app.config.ts
providers: [provideClientHydration(withIncrementalHydration())]
```

```
@defer (hydrate on viewport) {
  <app-comentarios />
}
```

Modo de renderizado por ruta

No todas las páginas necesitan lo mismo. Angular permite elegir **por ruta** cómo se renderiza:

- `Prerender`: HTML estático en compilación (contenido fijo).
- `Server`: HTML generado en cada petición (contenido dinámico y público, como el detalle de un producto).
- `Client`: renderizado solo en navegador (zonas privadas, paneles).

```
// app.routes.server.ts
export const serverRoutes: ServerRoute[] = [
  { path: '', renderMode: RenderMode.Prerender },
  { path: 'productos/:id', renderMode: RenderMode.Server },
  { path: 'panel/**', renderMode: RenderMode.Client },
];
```

Código seguro en el servidor

En el servidor **no existen** `window`, `document` ni `localStorage`. Si tu código los usa al crear un componente, la app fallará al renderizarse en el servidor. `afterNextRender()` ejecuta código **solo en el navegador**, después de que el componente se haya pintado, por lo que es el lugar adecuado para acceder a APIs del navegador o inicializar librerías que dependen del DOM.

```
export class Tema {
  tema = signal('claro');

  constructor() {
    afterNextRender(() => {
      this.tema.set(localStorage.getItem('tema') ?? 'claro');
    });
  }
}
```

Prácticas del capítulo 19

Práctica 1 · Básico — Activar SSR Crea un proyecto con `ng new mi-web --ssr` (genera el proyecto ya preparado para renderizar en servidor). Arráncalo, abre el código fuente de la página (Ctrl+U) y comprueba que el HTML ya contiene el contenido, a diferencia de una app sin SSR.

Práctica 2 · Intermedio — Modos de renderizado Crea una portada, una página "Sobre nosotros", un detalle de producto con parámetro y un panel privado. Configura en `app.routes.server.ts` el modo adecuado para cada ruta (`Prerender`, `Server` o `Client`) y justifica cada elección.

Práctica 3 · Reto — Código seguro en el servidor Añade al proyecto un componente que lea el tema guardado en `localStorage` al iniciar. Comprueba el error que se produce al renderizar en servidor y corrígelo con `afterNextRender`. Después, activa la hidratación incremental y convierte la sección de comentarios en un bloque `@defer (hydrate on viewport)`.

---

20. Optimización del rendimiento

Una aplicación rápida mejora la experiencia de uso y los resultados de negocio. Este capítulo reúne las técnicas principales para cargar menos, pintar menos y medir el resultado.

Carga diferida

La mejor optimización es **no cargar lo que todavía no se necesita**. Combina dos herramientas ya vistas: lazy loading de rutas (cada sección en su propio archivo) y `@defer` para todo lo que esté por debajo de la primera pantalla o detrás de una interacción (modales, gráficas, comentarios).

```
<app-cabecera />
<app-resumen />

@defer (on viewport) { <app-graficas /> } @placeholder { <div class="hueco"></div>
}
@defer (on interaction) { <app-editor-avanzado /> } @placeholder {
<button>Editar</button> }
```

Presupuestos de bundle

Los **budgets** son límites de tamaño que configuras en `angular.json`. Si la compilación supera el límite, `ng build` avisa o falla. Así evitas que la app engorde poco a poco sin que nadie lo note, por ejemplo al añadir una librería pesada.

```
"budgets": [
  { "type": "initial", "maximumWarning": "500kB", "maximumError": "1MB" },
  { "type": "anyComponentStyle", "maximumWarning": "4kB" }
]
```

NgOptimizedImage

Las imágenes suelen ser lo que más pesa en una web. La directiva `ngSrc` aplica **buenas prácticas automáticamente**: carga diferida de imágenes fuera de pantalla, reserva del espacio para evitar saltos de diseño (obligando a indicar `width` y `height`) y prioridad para la imagen principal con `priority`.

```
import { NgOptimizedImage } from '@angular/common';

@Component({
  imports: [NgOptimizedImage],
  template: `
    <img ngSrc="/fotos/portada.jpg" width="1200" height="600" priority />
    <img ngSrc="/fotos/producto.jpg" width="300" height="300" />
  `,
})
export class Portada {}
```

Virtual scroll

Pintar miles de elementos en el DOM bloquea el navegador. El **virtual scroll** del CDK solo crea los elementos **visibles en pantalla** (más un pequeño margen) y los recicla al hacer scroll. La lista se comporta como si tuviera todos los elementos, pero el DOM solo contiene unas decenas. `itemSize` es la altura de cada fila en píxeles.

```
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="48" style="height: 400px">
      <div *cdkVirtualFor="let cliente of clientes()">{{ cliente.nombre }}</div>
    </cdk-virtual-scroll-viewport>
  `,
})
export class ListaClientes {
  clientes = input.required<Cliente[]>(); // 10.000 elementos sin problema
}
```

Plantillas eficientes

Una función llamada desde la plantilla **se ejecuta en cada revisión** de la vista, aunque nada haya cambiado. Si es costosa, ralentiza la app. Las alternativas son un `computed` (se cachea hasta que cambian sus dependencias) o un **pipe puro** (solo se recalcula cuando cambia su entrada). Y recuerda usar siempre un `track` con id único en `@for`.

```
// ❌ Se recalcula en cada revisión
// <p>{{ calcularEstadisticas(ventas) }}</p>

// ✅ Con computed
estadisticas = computed(() => calcularEstadisticas(this.ventas()));

// ✅ Con un pipe puro (puros por defecto)
@Pipe({ name: 'iniciales' })
export class InicialesPipe implements PipeTransform {
  transform(nombre: string): string {
    return nombre.split(' ').map(p => p[0]).join('').toUpperCase();
  }
}
// <span>{{ usuario.nombre | iniciales }}</span>
```

Medir el rendimiento

Optimizar a ciegas suele ser perder el tiempo: primero **mide** dónde está el problema.

- **Angular DevTools** (extensión del navegador): la pestaña Profiler muestra qué componentes se revisan y cuánto tardan.
- **Lighthouse** (en Chrome DevTools): puntúa carga, accesibilidad y buenas prácticas.
- **Análisis del bundle**: muestra qué librerías ocupan más espacio.

```
ng build --stats-json
npx esbuild-visualizer --metadata dist/mi-app/stats.json --open
```

Prácticas del capítulo 20

Práctica 1 · Básico — Diagnóstico Pasa Lighthouse (F12 → Lighthouse) a una de tus aplicaciones compilada con `ng build` y servida en local. Anota la puntuación de rendimiento y los tres problemas principales que señala.

Práctica 2 · Intermedio — Aplicar mejoras En la misma aplicación: convierte las imágenes a `NgOptimizedImage` con `width`, `height` y `priority` en la principal, carga las rutas secundarias con lazy loading, envuelve con `@defer` lo que no se ve al principio y configura un budget en `angular.json`. Vuelve a pasar Lighthouse y compara.

Práctica 3 · Reto — Listas enormes Genera una lista de 10.000 contactos y muéstrala primero con `@for` normal y después con el virtual scroll del CDK. Mide con el Profiler de Angular DevTools el tiempo de carga y

de scroll en ambos casos. Sustituye además una función llamada desde la plantilla por un `computed` o un pipe puro y comprueba la diferencia.

---

21. Arquitectura de aplicaciones

La arquitectura determina si un proyecto sigue siendo fácil de mantener cuando crece en código y en equipo. Este capítulo explica cómo organizar carpetas, componentes, repositorios y librerías.

Arquitectura por funcionalidades

En proyectos grandes, organizar por tipo de archivo (`components/`, `services/`) acaba en carpetas enormes sin relación entre sí. Es mejor **organizar por funcionalidad** (features), y dentro de cada una separar capas:

- `core/`: lo global y único (auth, interceptores, layout).
- `shared/`: piezas reutilizables sin lógica de negocio.
- `features/<nombre>/data`: acceso a datos (servicios API, stores).
- `features/<nombre>/ui`: componentes presentacionales.
- `features/<nombre>/feature`: componentes contenedores y páginas.

```
src/app/
├── core/
├── shared/
└── features/
    ├── productos/
    │   ├── data/
    │   ├── ui/
    │   ├── feature/
    │   └── productos.routes.ts
    └── carrito/
```

Componentes contenedores y presentacionales

Separar responsabilidades hace los componentes más reutilizables y fáciles de testear:

- **Contenedores** (o "smart"): inyectan servicios, obtienen datos y deciden qué hacer ante los eventos.
- **Presentacionales** (o "dumb"): solo reciben `input()` y emiten `output()`. No saben de dónde vienen los datos ni qué pasará con sus eventos.

```
// Presentacional: reutilizable en cualquier sitio
@Component({
  selector: 'app-lista-productos',
  template: `
    @for (p of productos(); track p.id) {
      <app-tarjeta-producto [producto]="p" (anadir)="anadir.emit($event)" />
    }
  `,
})
export class ListaProductosUi {
```

```
  productos = input.required<Producto[]>();
  anadir = output<number>();
}

// Contenedor: conecta datos y acciones
@Component({
  imports: [ListaProductosUi],
  template: `<app-lista-productos [productos]="productos.value() ?? []"
(anadir)="carrito.anadirPorId($event)" />`,
})
export class PaginaProductos {
  productos = httpResource<Producto[]>(() => '/api/productos');
  carrito = inject(Carrito);
}
```

Monorepos con Nx

Un **monorepo** contiene varias aplicaciones y librerías en un único repositorio, de modo que comparten código sin publicar paquetes. **Nx** es la herramienta más usada para gestionarlos en Angular: genera librerías, impone **reglas de dependencia** entre capas (por ejemplo, que `ui` no pueda importar de `data`) y ejecuta builds y tests **solo de lo afectado** por un cambio.

```
npx create-nx-workspace@latest mi-org --preset=angular-monorepo
nx g @nx/angular:library --name=productos-data --directory=libs/productos/data
nx affected -t test        # solo testea lo que ha cambiado
nx graph                   # visualiza dependencias entre proyectos
```

Librerías propias

Una librería agrupa componentes, servicios o utilidades para **reutilizarlos en varios proyectos**, por ejemplo un sistema de diseño de la empresa. Angular CLI puede generarla y compilarla en formato publicable en npm (o en un registro privado). Lo que se exporta públicamente se declara en `public-api.ts`.

```
ng generate library ui-kit
ng build ui-kit
```

```
// projects/ui-kit/src/public-api.ts
export * from './lib/boton/boton';
export * from './lib/tarjeta/tarjeta';
```

Micro-frontends

Consisten en dividir una aplicación grande en **varias apps independientes**, desarrolladas y desplegadas por separado, que se componen en una sola en el navegador. Con Angular se suelen implementar con Native

Federation o Module Federation. Aportan independencia entre equipos, pero añaden mucha complejidad: solo merecen la pena con **equipos grandes y autónomos** que trabajan en partes muy separadas.

```
// En la app "shell", una ruta carga otra app desplegada por separado
import { loadRemoteModule } from '@angular-architects/native-federation';

export const routes: Routes = [
  {
    path: 'facturacion',
    loadChildren: () => loadRemoteModule('facturacion', './routes').then(m =>
m.routes),
  },
];
```

Prácticas del capítulo 21

Práctica 1 · Básico — Reorganizar por funcionalidades Toma una de tus prácticas con varias pantallas y reorganiza las carpetas en `core/`, `shared/` y `features/`, con las subcarpetas `data`, `ui` y `feature` dentro de cada funcionalidad. Comprueba que la aplicación sigue compilando.

Práctica 2 · Intermedio — Contenedor y presentacional Divide una página de listado en un componente contenedor (que inyecte el servicio y gestione las acciones) y un componente presentacional (que solo tenga `input()` y `output()`). Reutiliza el componente presentacional en otra página con datos distintos.

Práctica 3 · Reto — Librería propia Crea una librería `ui-kit` con `ng generate library ui-kit` (genera un proyecto de librería dentro del workspace) que contenga un botón y una tarjeta con tu propio estilo. Expórtalos en `public-api.ts`, compílala con `ng build ui-kit` (genera la versión distribuible en `dist/`) y úsala desde la aplicación principal.

---

22. Accesibilidad

Una aplicación accesible puede usarla cualquier persona, independientemente de cómo interactúe con ella. Este capítulo resume las prácticas básicas en Angular.

Accesibilidad

Una aplicación accesible puede usarse con **teclado, lector de pantalla y otras tecnologías de apoyo**. Además de ser una obligación legal en muchos contextos, mejora la experiencia de todos. Claves:

- HTML semántico (`button`, `nav`, `main`, `label`) antes que `div` con eventos.
- Atributos ARIA solo cuando el HTML no basta.
- Gestión del foco en modales y cambios de ruta (CDK `FocusTrap`, `LiveAnnouncer`).
- Contraste suficiente y navegación completa por teclado.

```
<!-- ❌ No accesible: no se enfoca con teclado ni se anuncia como botón -->
<div (click)="abrir()">Menú</div>
```

```
<!-- ✅ Accesible -->
<button type="button" (click)="abrir()" [attr.aria-expanded]="abierto()" aria-
controls="menu">
  Menú
</button>
<ul id="menu" [hidden]="!abierto()">…</ul>

<label for="email">Email</label>
<input id="email" type="email" />
```

Prácticas del capítulo 22

Práctica 1 · Básico — Navegar sin ratón Usa una de tus aplicaciones solo con el teclado (Tab, Shift+Tab, Enter, Espacio). Anota los elementos a los que no se puede llegar o que no se pueden activar y corrígelos usando los elementos HTML adecuados (`button`, `a`, `label`).

Práctica 2 · Intermedio — Formularios y ARIA Revisa un formulario: asocia cada `input` con su `label`, anuncia los mensajes de error con `role="alert"` y añade `[attr.aria-expanded]` a un botón que abra y cierre un menú. Pasa la auditoría de accesibilidad de Lighthouse antes y después.

Práctica 3 · Reto — Lector de pantalla Activa el lector de pantalla del sistema (NVDA en Windows o VoiceOver en macOS) y recorre la aplicación. Crea un diálogo modal que atrape el foco con el CDK (`cdkTrapFocus`), lo devuelva al botón que lo abrió al cerrarse y anuncie las acciones importantes con `LiveAnnouncer`.

---

23. Internacionalización y seguridad

Este capítulo explica cómo preparar la aplicación para varios idiomas y cómo protegerla de los ataques más comunes en el navegador.

Internacionalización (i18n)

Preparar la app para **varios idiomas**. El sistema integrado marca los textos con el atributo `i18n`, los extrae a un archivo de traducción y genera **una compilación por idioma**. Es muy eficiente, pero cambiar de idioma implica cargar otra versión de la app. Si necesitas cambiar de idioma en caliente sin recargar, se usan librerías como Transloco.

```
<h1 i18n="@@tituloInicio">Bienvenido</h1>
<p i18n>Tienes {count, plural, =0 {ningún mensaje} =1 {un mensaje} other
{{{count}} mensajes}}</p>
```

```
ng extract-i18n          # genera messages.xlf con los textos
ng build --localize      # una versión compilada por idioma
```

Seguridad

Angular protege de serie contra los ataques más comunes, pero conviene conocer sus límites:

- **XSS**: Angular escapa todo lo que interpolas. Evita insertar HTML no confiable con `innerHTML` y no uses `bypassSecurityTrust…` sin estar seguro del origen.
- **XSRF**: `HttpClient` incluye protección integrada si el servidor la soporta.
- **Secretos**: nunca guardes claves privadas en el frontend; todo el código llega al navegador.
- **CSP** (Content Security Policy): configúrala en el servidor para limitar qué scripts pueden ejecutarse.

```
comentario = '<img src=x onerror="alert(1)">';
```

```
<!-- ✅ Se muestra como texto, no se ejecuta -->
<p>{{ comentario }}</p>

<!-- ⚠️ Angular lo sanea, pero evita esta vía con contenido de usuarios -->
<p [innerHTML]="comentario"></p>
```

Prácticas del capítulo 23

Práctica 1 · Básico — Marcar textos Marca con `i18n` y un identificador propio (`@@nombre`) todos los textos de una de tus prácticas, incluido un mensaje con plural ("1 tarea" / "N tareas"). Extráelos con `ng extract-i18n` (genera el archivo `messages.xlf` con todos los textos marcados).

Práctica 2 · Intermedio — Dos idiomas Crea la traducción al inglés (o al euskera) del archivo extraído, configura los dos idiomas en `angular.json` y genera las dos versiones con `ng build --localize` (compila una versión de la aplicación por cada idioma). Comprueba que fechas y números también cambian de formato.

Práctica 3 · Reto — Seguridad Crea un campo de comentarios y muestra lo que escribe el usuario con interpolación y con `[innerHTML]`. Prueba a escribir `<img src=x onerror="alert(1)">` y `<b>hola</b>` y explica qué hace Angular en cada caso. Busca en tus proyectos si hay alguna clave o contraseña escrita en el código del frontend.

---

24. Actualización y migraciones

Angular publica una versión mayor al año. Este capítulo explica cómo mantener un proyecto al día y cómo modernizar código antiguo de forma automática.

Actualizar con ng update

Mantener Angular al día te da mejoras de rendimiento, nuevas APIs y parches de seguridad. `ng update` actualiza las dependencias y **ejecuta automáticamente las migraciones de código** necesarias. Actualiza **una versión mayor cada vez** y sigue la guía oficial (angular.dev/update-guide), que indica los pasos según tu versión de origen y destino.

```
ng update                                # muestra qué se puede actualizar
ng update @angular/core @angular/cli     # actualiza el núcleo
ng update @angular/material              # y las librerías relacionadas
```

Migraciones automáticas

Angular incluye **schematics de migración** que reescriben el código antiguo al estilo moderno de forma automática. Son ideales para modernizar proyectos heredados sin hacerlo a mano. Ejecútalos de uno en uno, revisa el resultado y pasa los tests entre cada paso.

```
ng generate @angular/core:standalone              # NgModules → standalone
ng generate @angular/core:control-flow            # *ngIf/*ngFor → @if/@for
ng generate @angular/core:signal-input-migration  # @Input → input()
ng generate @angular/core:output-migration        # @Output → output()
ng generate @angular/core:inject                  # constructor → inject()
```

Schematics propios

Un schematic es un **generador de código**. Igual que `ng g component`, puedes crear los tuyos para automatizar las convenciones de tu equipo: por ejemplo, generar una feature completa con sus carpetas `data/ui/feature`, rutas y tests. Garantizan consistencia y ahorran tiempo en proyectos grandes.

```
npm install -g @angular-devkit/schematics-cli
schematics blank --name=mis-schematics
```

```
// src/feature/index.ts
export function feature(options: { name: string }): Rule {
  return mergeWith(
    apply(url('./files'), [
      applyTemplates({ ...strings, ...options }),
      move(`src/app/features/${strings.dasherize(options.name)}`),
    ])
  );
}
```

Prácticas del capítulo 24

Práctica 1 · Básico — Comprobar actualizaciones En uno de tus proyectos, ejecuta `ng update` sin parámetros (muestra qué paquetes tienen versiones nuevas, sin cambiar nada). Consulta en angular.dev/update-guide los pasos para pasar de tu versión a la siguiente.

Práctica 2 · Intermedio — Modernizar código antiguo Busca un ejemplo antiguo de Angular en internet (con `NgModule`, `*ngIf`, `@Input()` y dependencias en el constructor) y cópialo en un proyecto. Ejecuta de una

en una las migraciones automáticas del capítulo (standalone, control flow, inputs, outputs e `inject`), revisando y compilando después de cada una.

Práctica 3 · Reto — Schematic propio Crea un schematic que genere una funcionalidad completa con sus carpetas `data`, `ui` y `feature`, un servicio con `@Service()`, un componente contenedor y su archivo de rutas. Úsalo para generar una funcionalidad nueva en uno de tus proyectos.

Repaso de la Parte IV

- [ ] Sé cuándo necesito un store y diseño uno con SignalStore
- [ ] Configuro SSR, prerender e hidratación incremental
- [ ] Mido y optimizo rendimiento con datos, no a ojo
- [ ] Estructuro proyectos grandes por features y capas
- [ ] Cuido accesibilidad, i18n y seguridad
- [ ] Migro un proyecto antiguo a Angular moderno

Proyectos prácticos

Practicar con proyectos reales es lo que consolida lo aprendido. Cada proyecto reutiliza lo anterior y añade lo nuevo de cada parte.

Parte	Proyecto	Qué practicas
I · Fundamentos	Lista de tareas local	Componentes, binding, control flow
II · Desarrollo de aplicaciones	Tienda con catálogo, detalle, carrito y login	Signals, servicios, router, formularios, HTTP
III · Angular en profundidad	Panel de administración con buscador, tablas y gráficas	RxJS, `httpResource`, Material, OnPush, tests
IV · Aplicaciones profesionales	La tienda con SSR, SignalStore, i18n y monorepo Nx	Arquitectura, rendimiento, despliegue

Para la API puedes usar `json-server` o una API pública como DummyJSON mientras no tengas backend propio.

```
npx json-server db.json   # API REST falsa a partir de un archivo JSON
```

Recursos

- **angular.dev**: documentación oficial y tutorial interactivo.
- **Blog oficial de Angular**: novedades de cada versión.
- **Angular DevTools**: extensión del navegador para inspeccionar componentes, signals y rendimiento.
- **Documentación de NgRx Signals**, **Nx** y **RxJS** (rxjs.dev, con su árbol de decisión de operadores).
- **Angular Update Guide**: angular.dev/update-guide.